

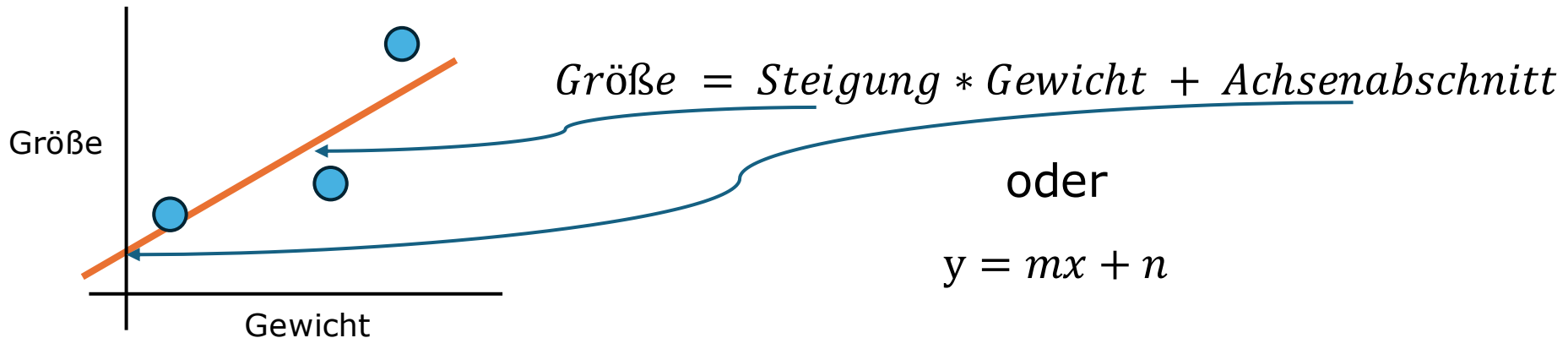
Gradientenabstieg am Beispiel neuronaler Netze

Orhan Konak
Hochschule Hannover
17.03.2026



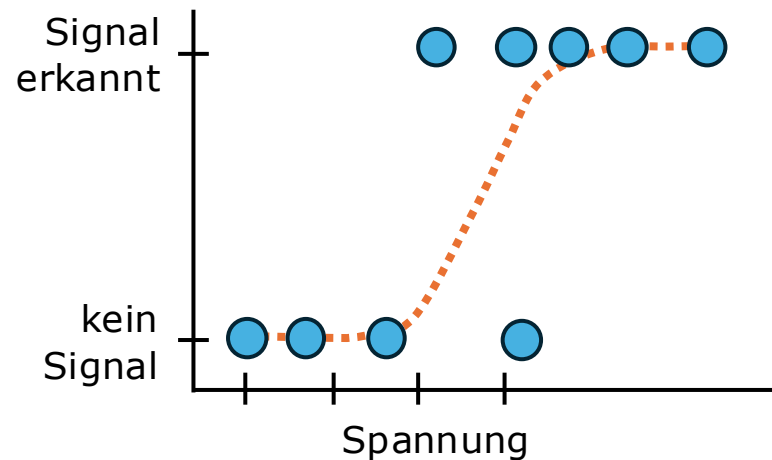
Motivation

- In Statistik, Machine Learning und Data Science lösen wir häufig **Optimierungsprobleme**
- Wenn wir z.B. eine Ausgleichsgerade mit **Linear Regression** berechnen, optimieren wir den **Achsenabschnitt** und die **Steigung**

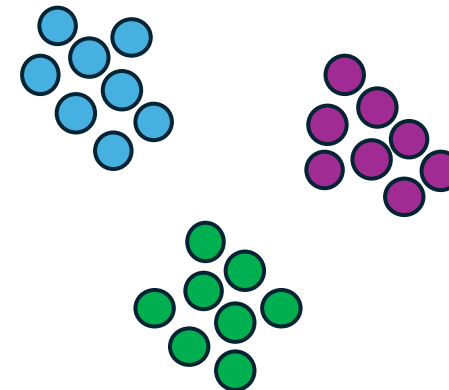


Motivation

Wenn wir **Logistic Regression** nutzen, optimieren wir eine **S-förmige Kurve**

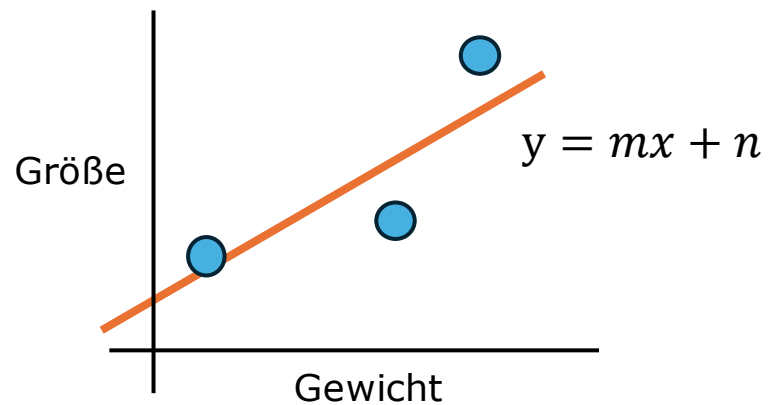


... oder mit **t-SNE** optimieren wir **Cluster**

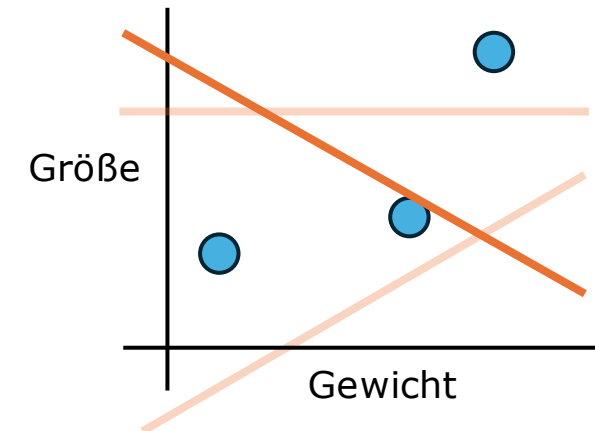
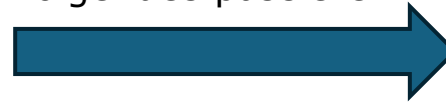


Motivation

- **Frage:** *Doch wie können wir beurteilen, wie gut unser Modell ist?*



Durch das Ändern
von m und n
könnten auch
folgendes passieren



- **Antwort:** *Mit dem **Gradientenabstieg** können wir viele dieser Optimierungsprobleme lösen*

Agenda

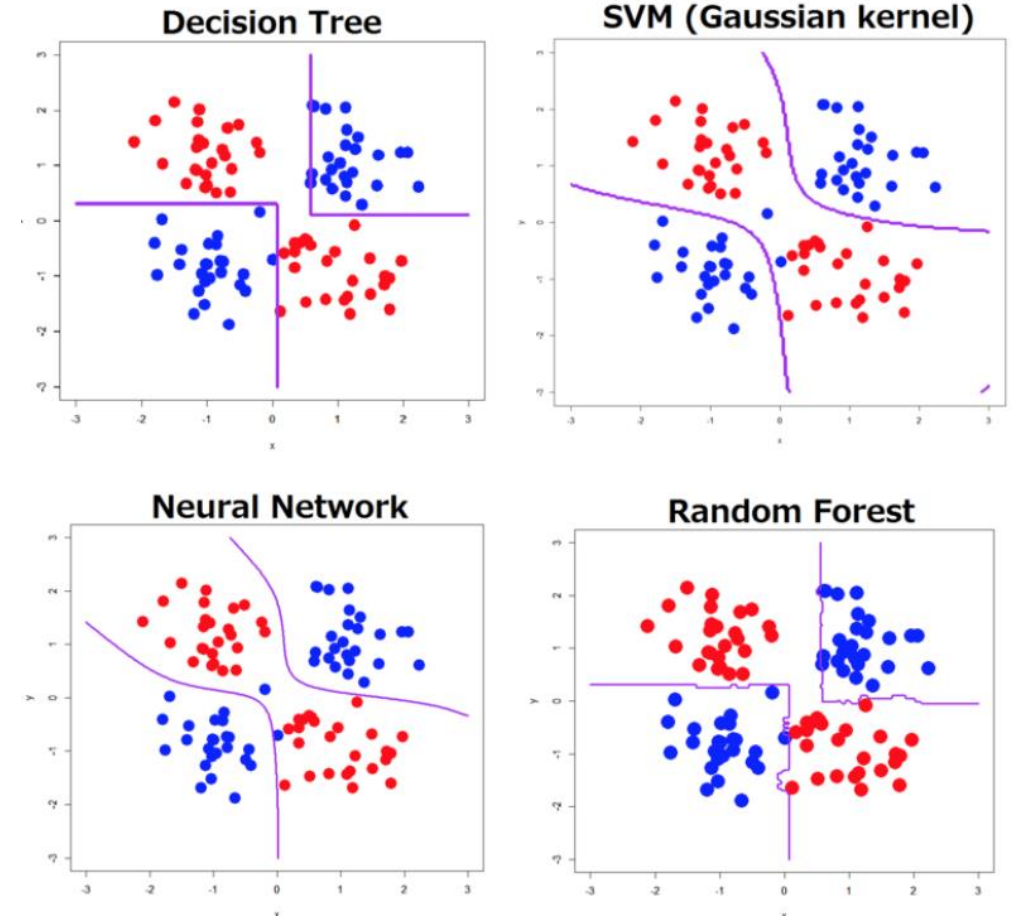
- Neuronale Netze
- Gradientenabstieg
- Beispiel

Agenda

- **Neuronale Netze**
- Gradientenabstieg
- Beispiel

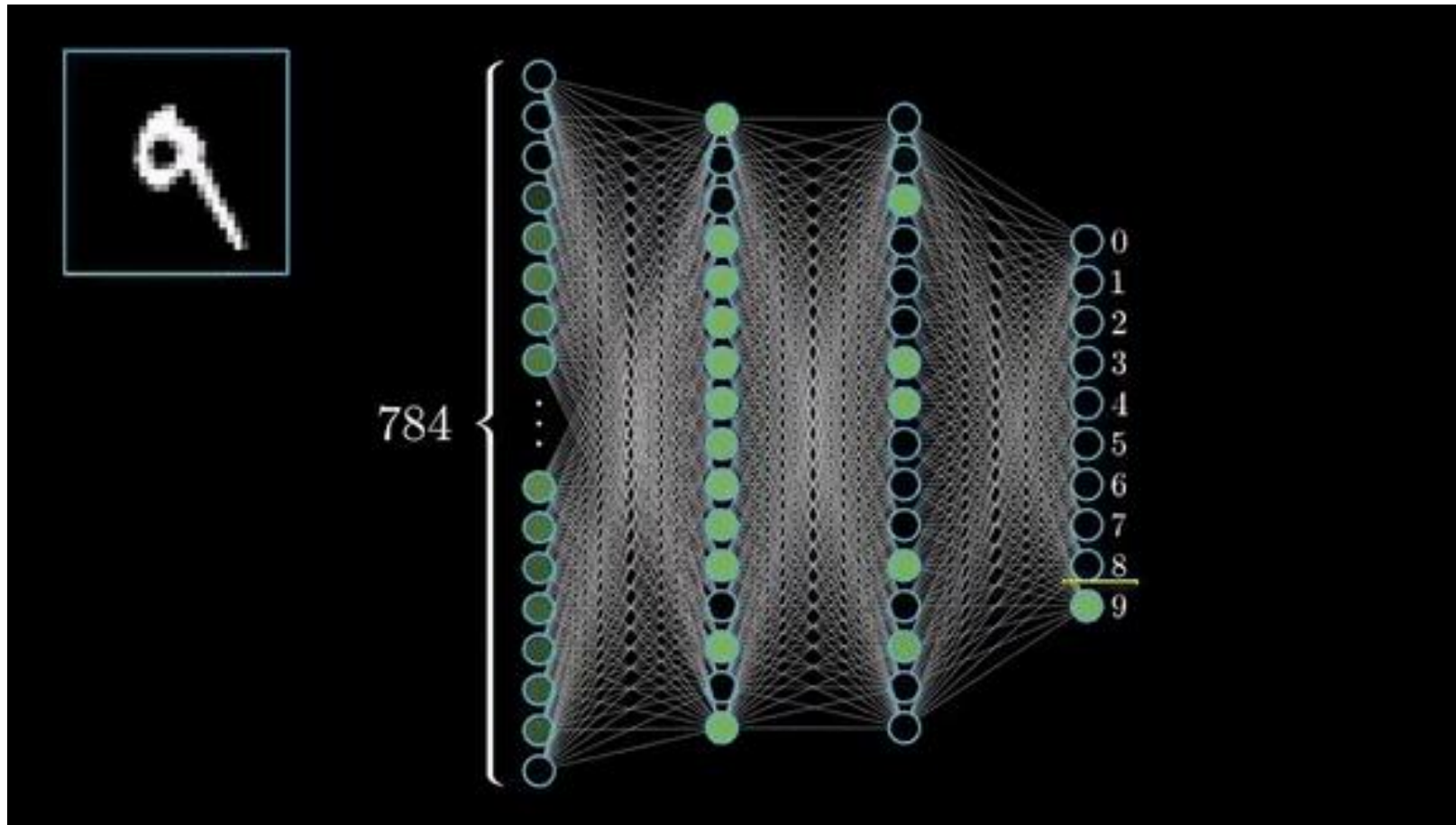
Warum Neuronale Netze?

- Viele Machine-Learning-Aufgaben erfordern die Modellierung **nichtlinearer Zusammenhänge**
- **Lineare Modelle** sind dafür häufig **nicht ausreichend**
- **Neuronale Netze** sind **universelle Funktionsapproximatoren**
- Einsatz z. B. in
 - **Allgemein:** Computer Vision, Natural Language Processing, Medizin oder Zeitreihenprognosen
 - **Elektrotechnik:** Signalerkennung in Kommunikationssystemen, Fehlerdiagnose in elektrischen Maschinen, Prognose von Netzlast in Smart Grids
- **Frage:** *Wie lernen neuronale Netze eigentlich?*



https://www.linkedin.com/posts/swathi-sreejith-2bb899275_machinelearning-datascience-artificialintelligence-activity-7383420093478637569-CQNI

Neuronales Netz Beispiel

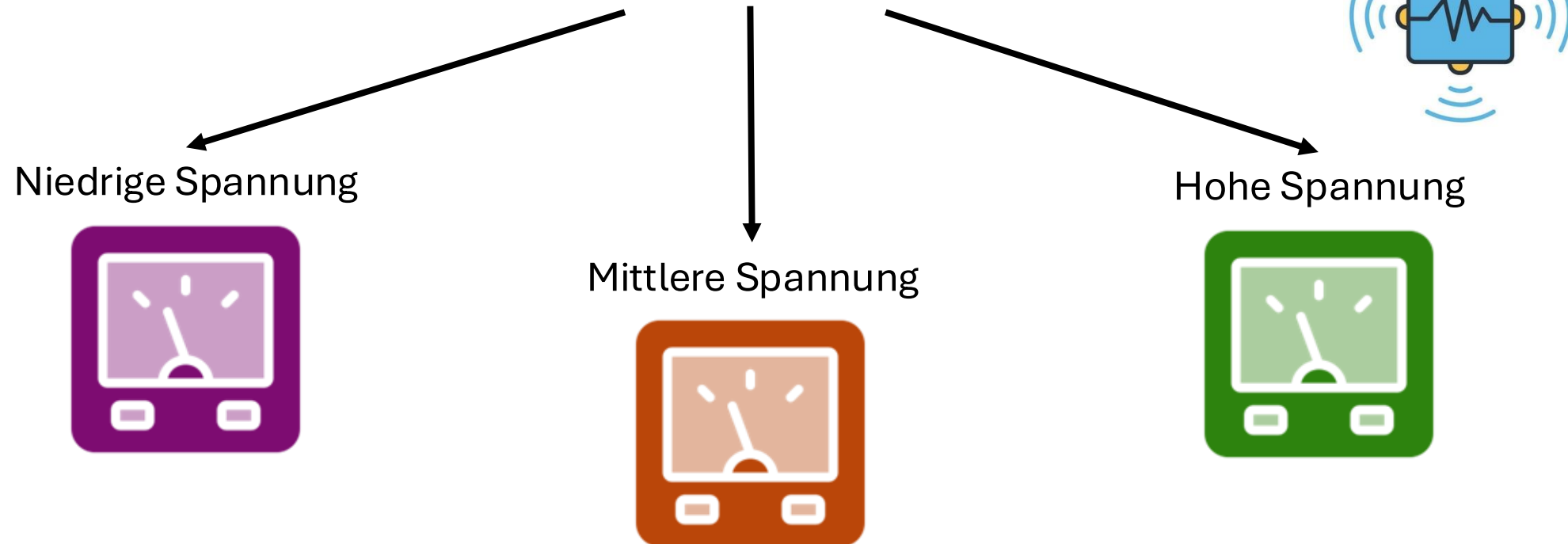


<https://medium.com/analytics-vidhya/applying-ann-digit-and-fashion-mnist-13accfc44660>

Neuronales Netz Beispiel

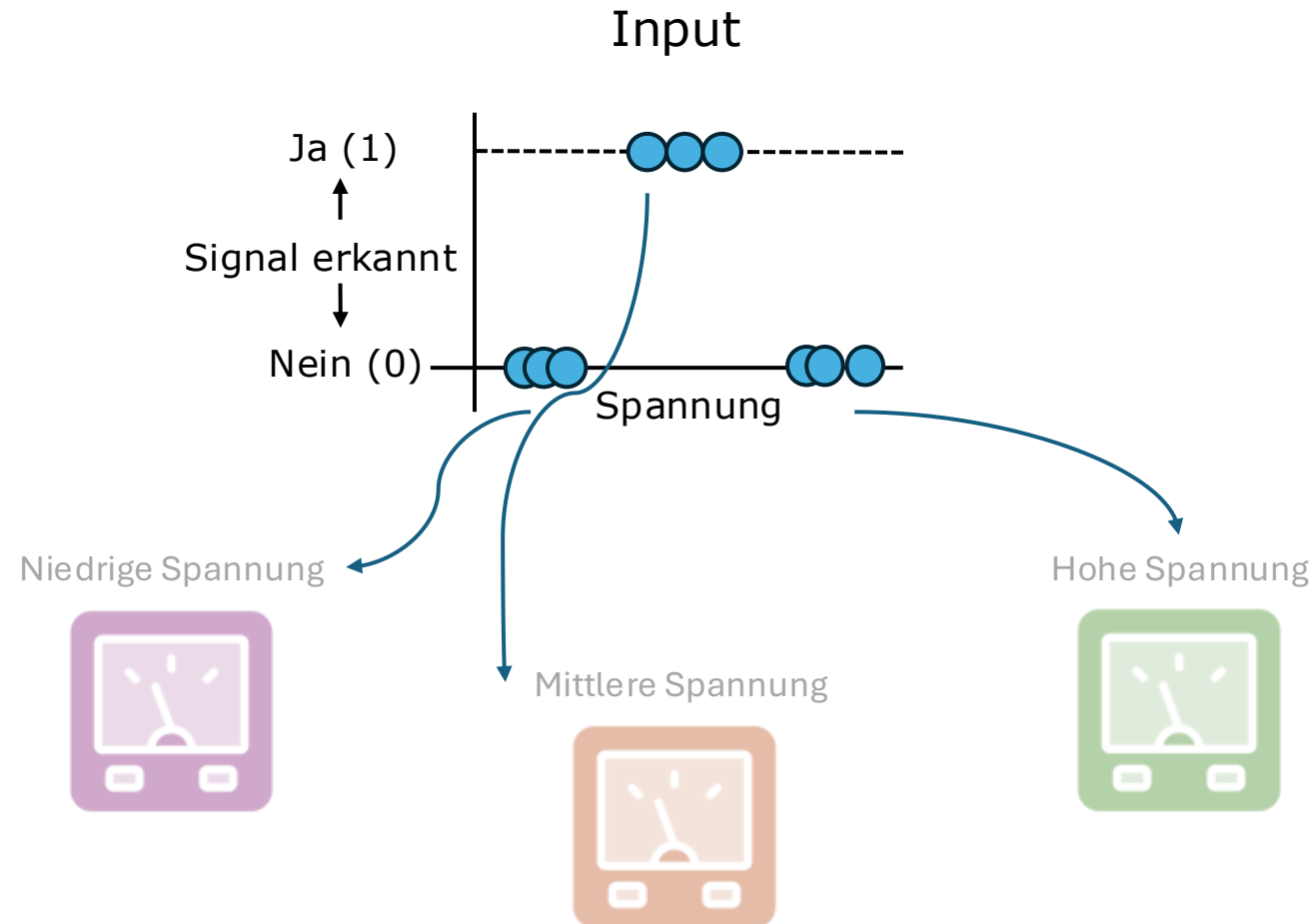
■ Beispiel: **Sensorsignal**

Wir legen **drei unterschiedliche Spannungen** an einen elektronischen Sensor an. Für jede Spannung beobachten wir, ob der Sensor ein **Signal erkennt oder nicht**.

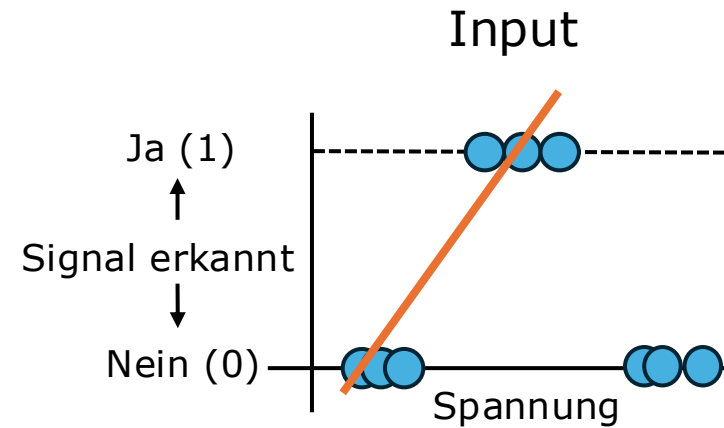


Neuronales Netz Beispiel

Wir können die Messpunkte im Diagramm darstellen

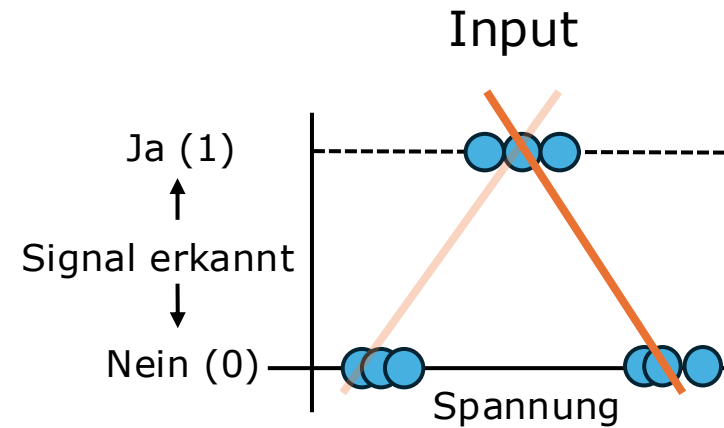


Neuronales Netz Beispiel



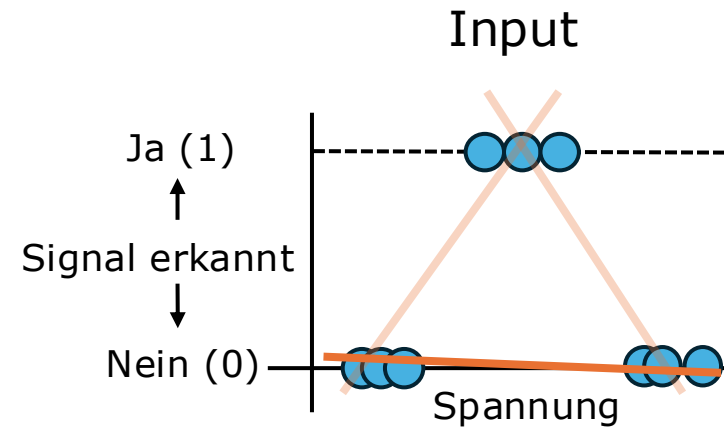
Wir können jedoch nicht einfach eine **Gerade** an die Daten anpassen, um Vorhersagen zu treffen ...

Neuronales Netz Beispiel



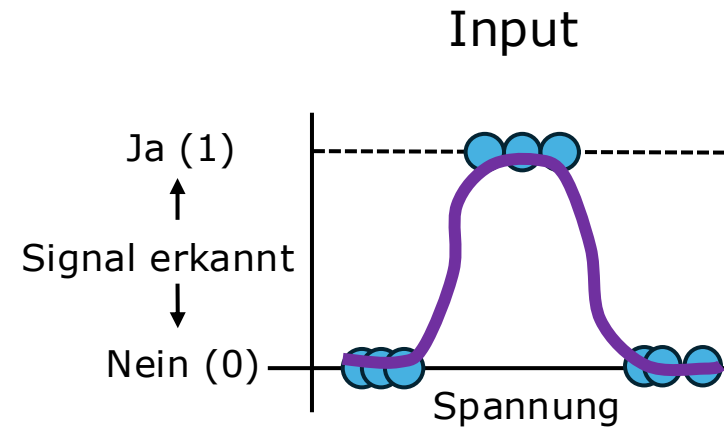
Wir können jedoch nicht einfach eine **Gerade** an die Daten anpassen, um Vorhersagen zu treffen ...

Neuronales Netz Beispiel



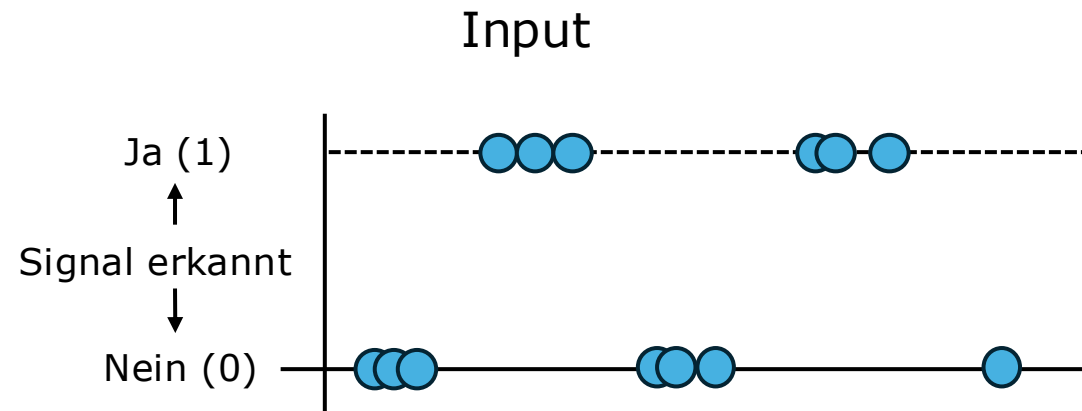
Wir können jedoch nicht einfach eine **Gerade** an die Daten anpassen, um Vorhersagen zu treffen ...

Neuronales Netz Beispiel



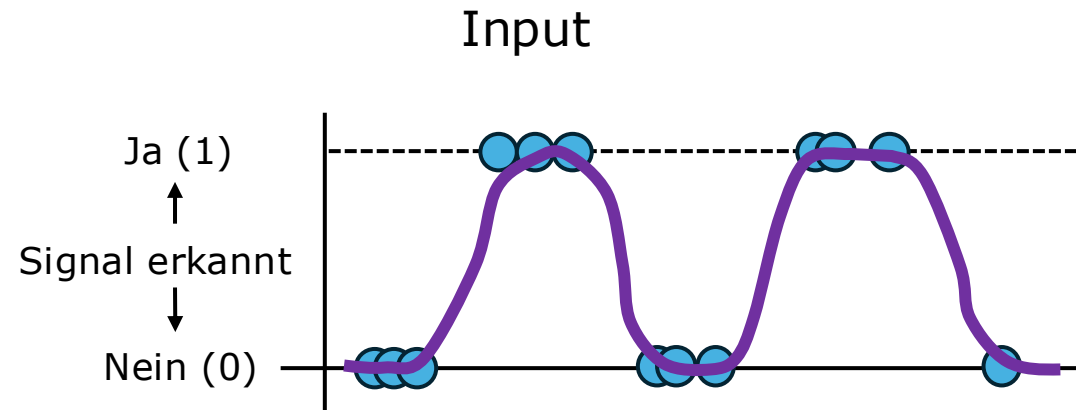
Die gute Nachricht: Ein **neuronales Netz** kann auch **komplexe, nichtlineare Zusammenhänge** in den Daten **modellieren**

Neuronales Netz Beispiel



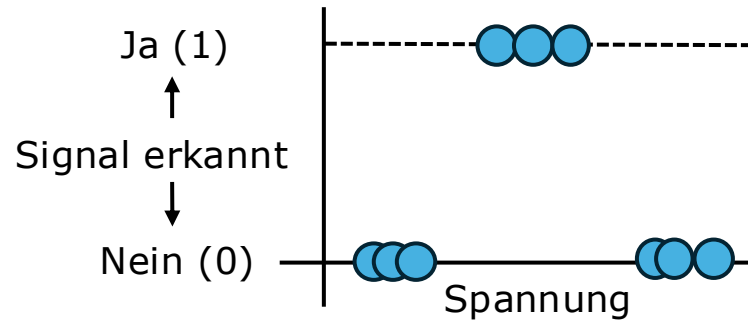
... selbst bei sehr
komplexen Datensätzen
wie diesem

Neuronales Netz Beispiel



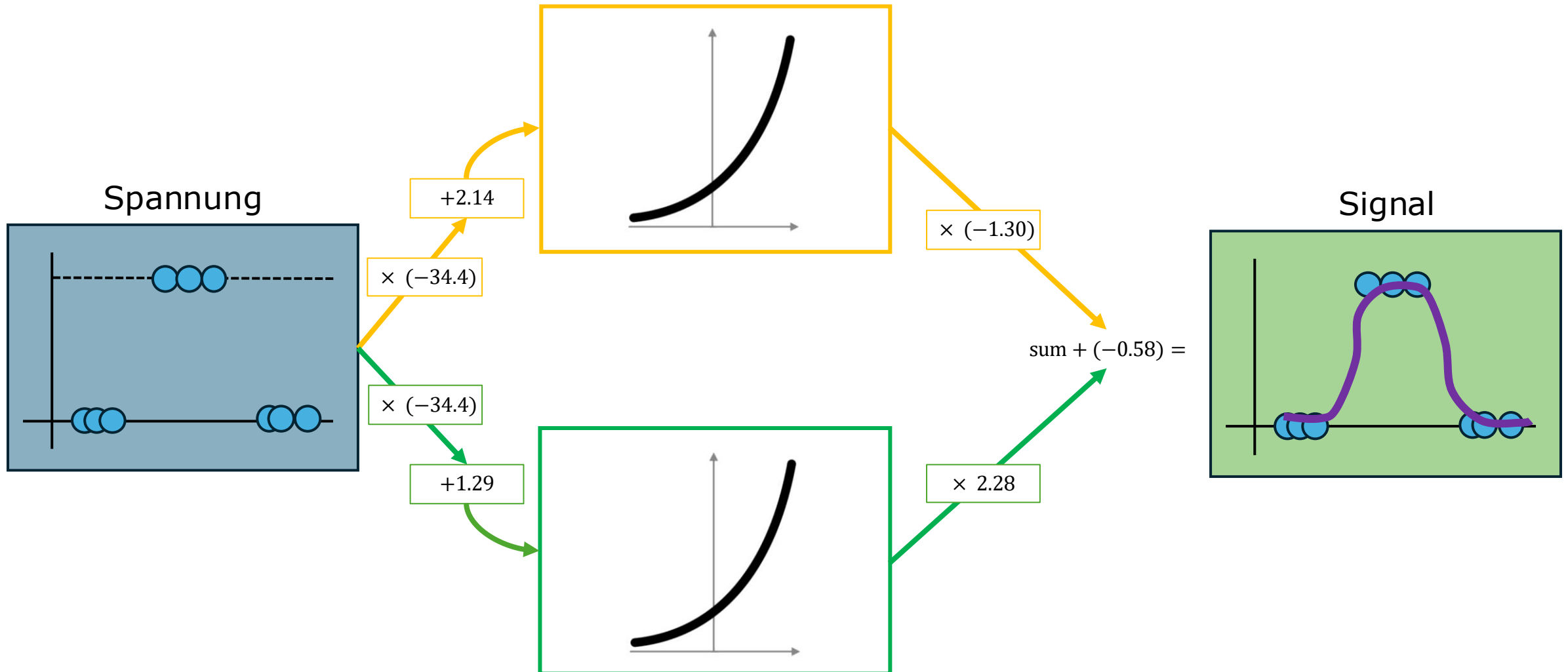
... selbst bei sehr
komplexen Datensätzen
wie diesem

Neuronales Netz Beispiel

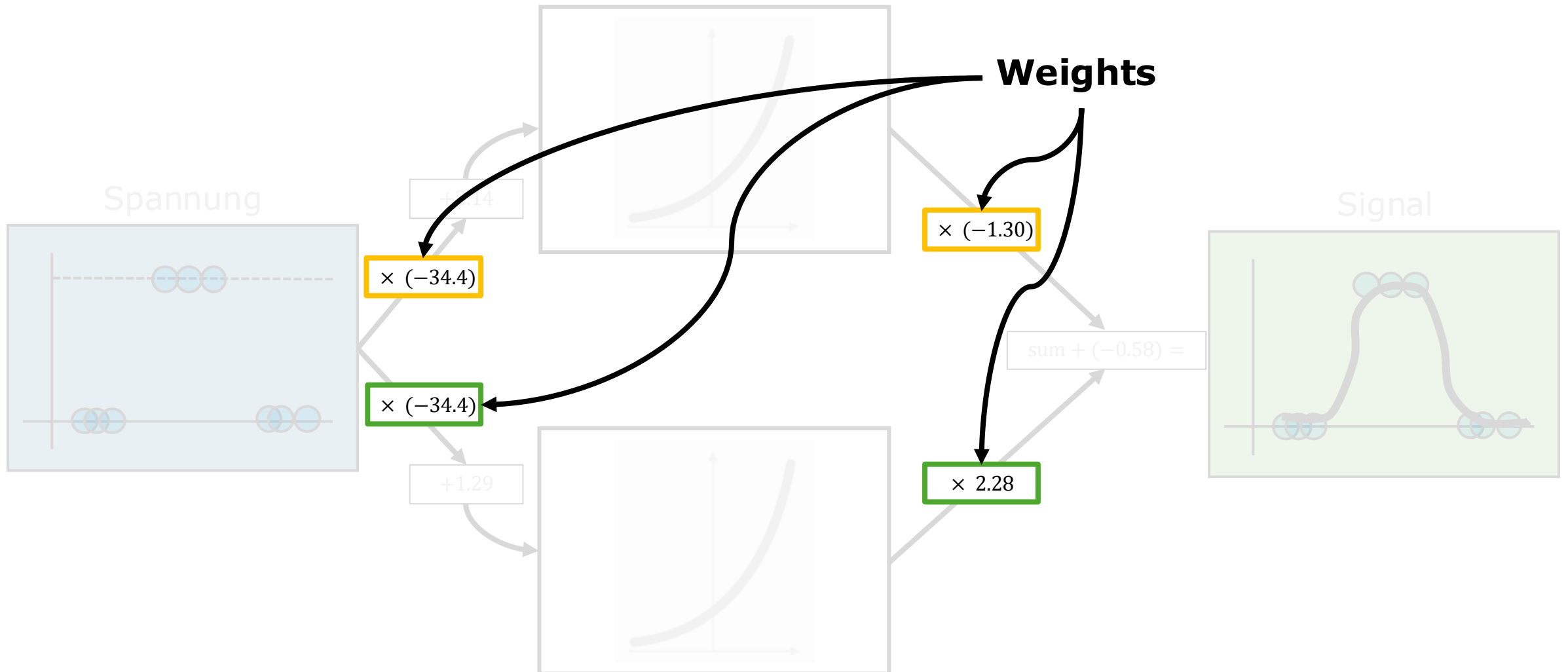


Wir können diesen einfachen Beispieldatensatz verwenden und daraus ein **neuronales Netz** modellieren

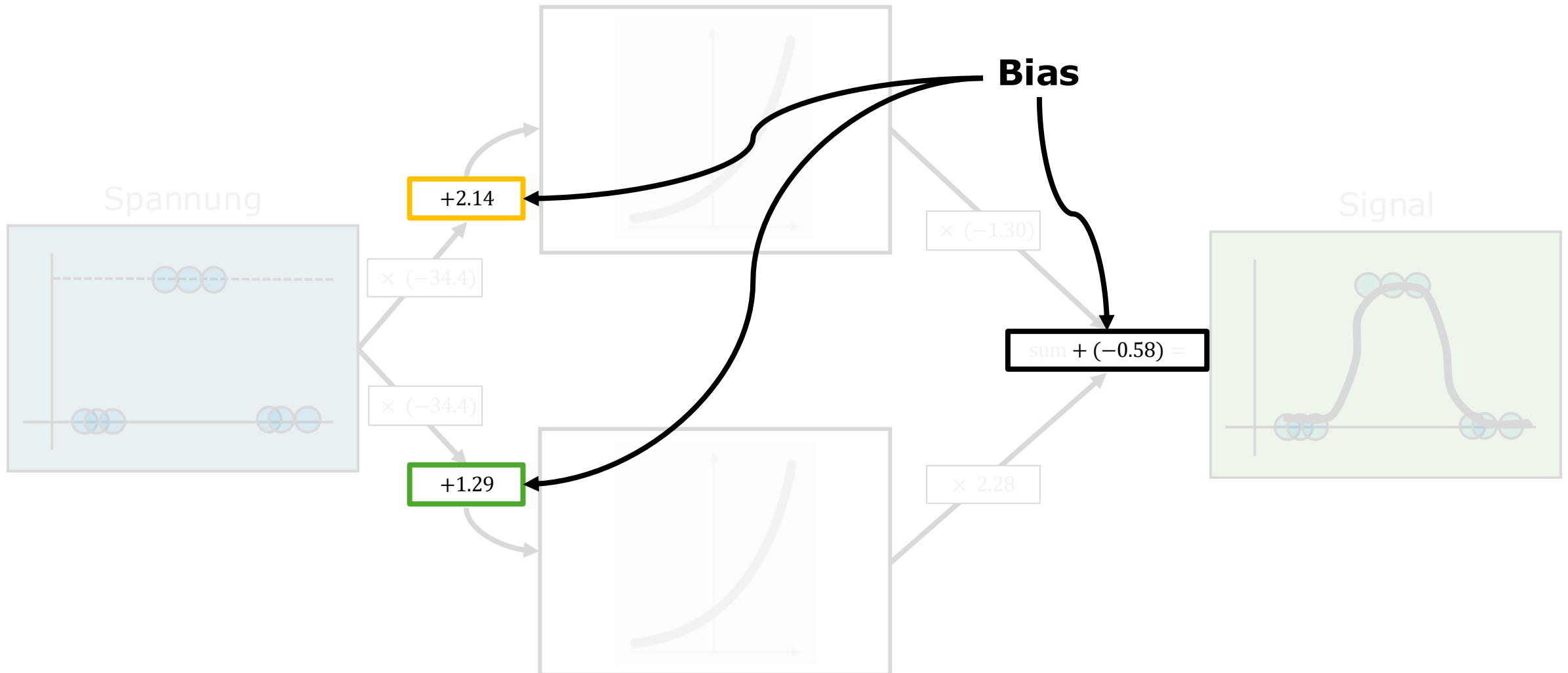
Neuronales Netz Beispiel



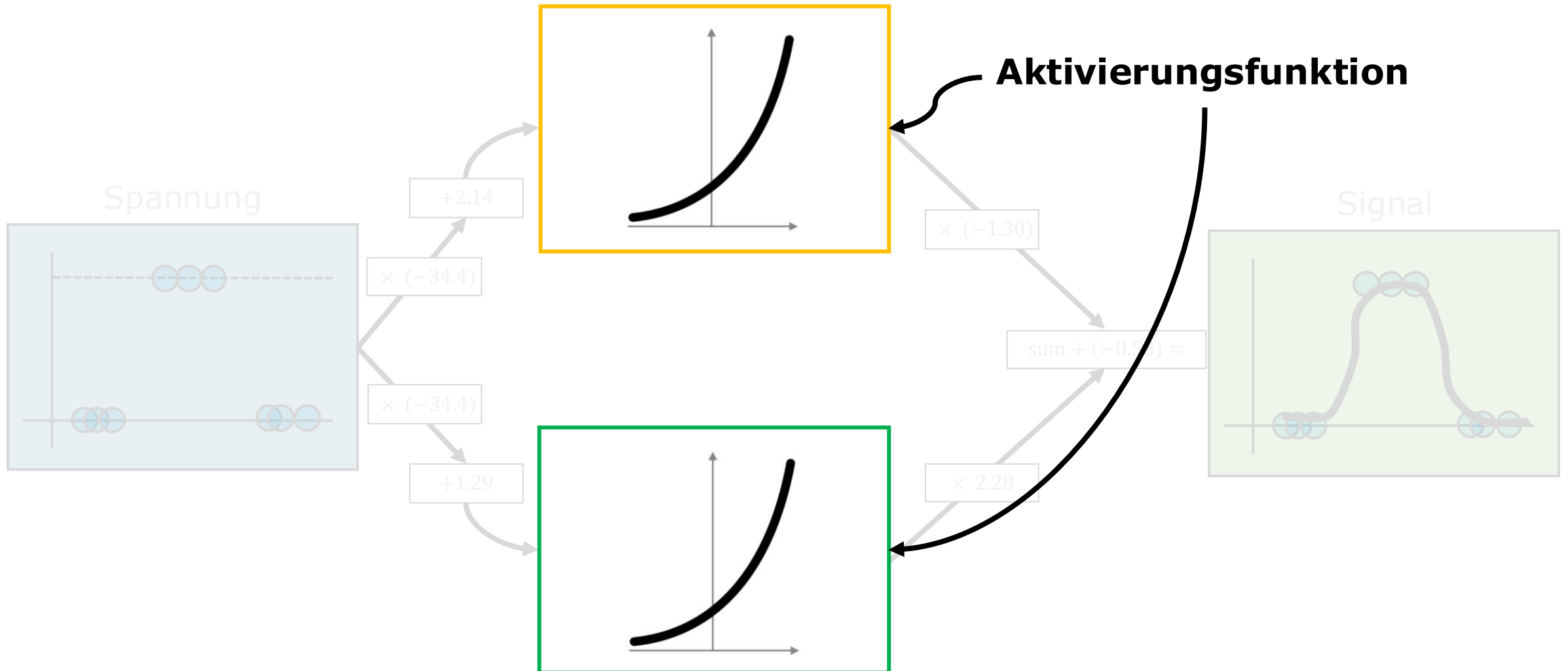
Neuronales Netz Beispiel



Neuronales Netz Beispiel



Neuronales Netz Beispiel



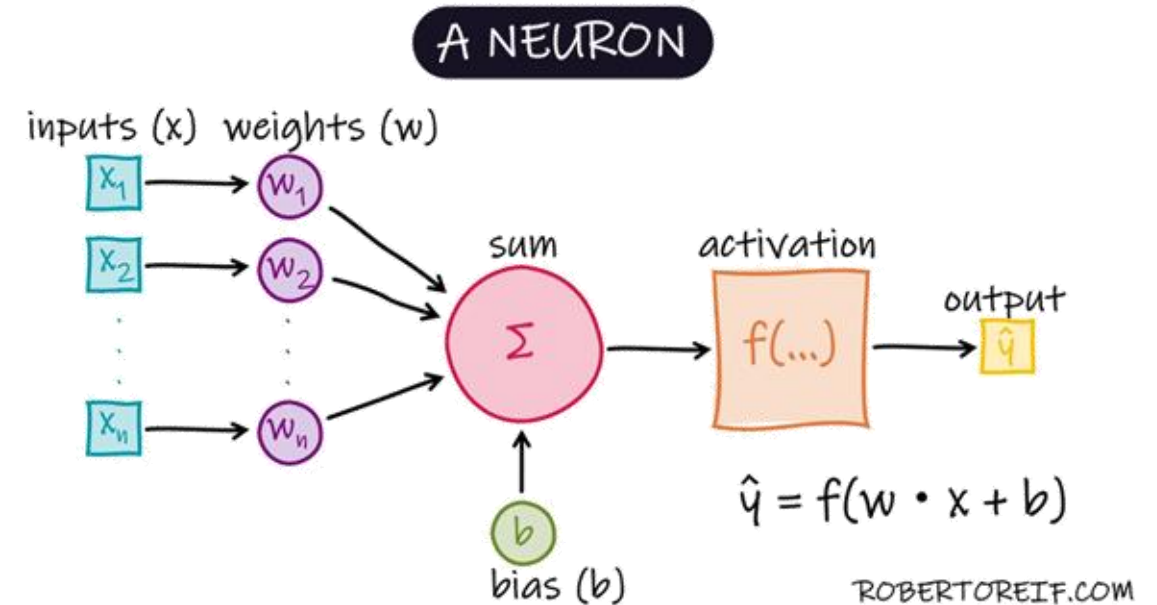
Neuronales Netz Definition

In mathematischer Notation:

Für ein einzelnes Neuron gilt:

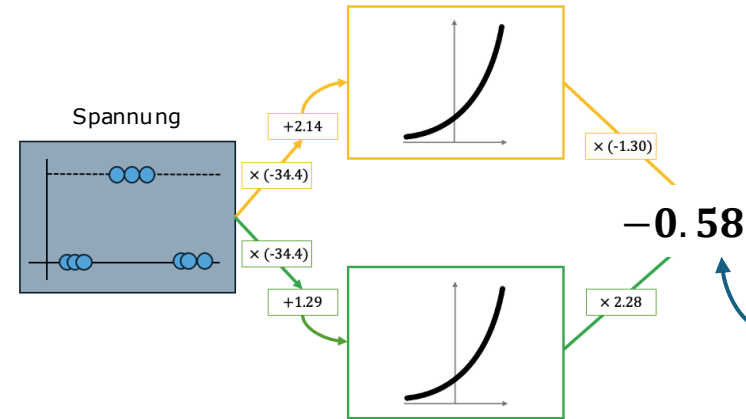
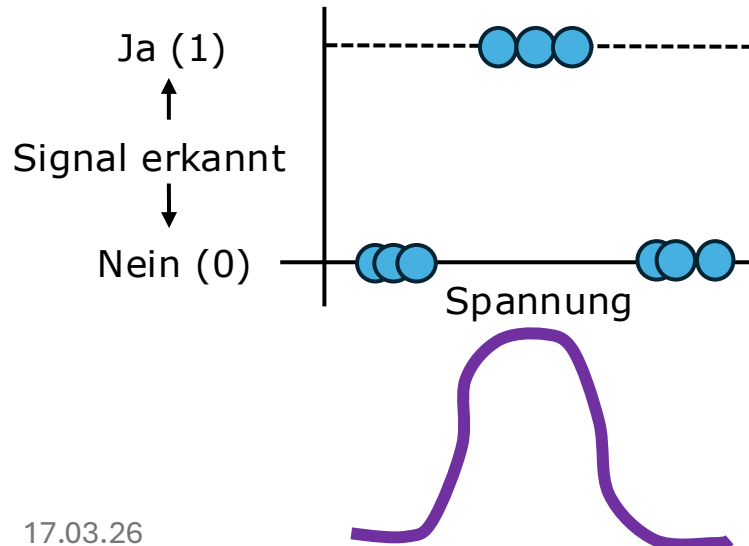
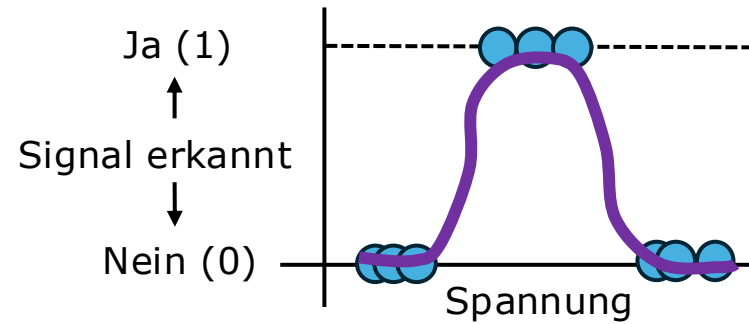
$$z = w^T x + b$$

- x : Eingabevektor
- w : Gewichtsvektor
- b : Bias (Verschiebung)
- z : lineare Kombination der Eingaben

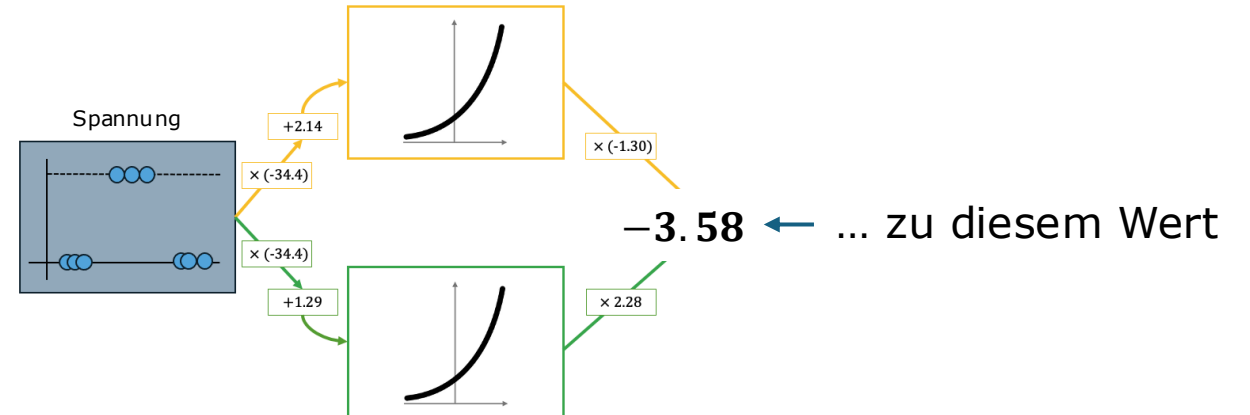


- **Mehrere Schichten stapeln → Deep Neural Network**
- **Frage:** *Wie können wir die **Gewichte** w und den **Bias** b berechnen bzw. optimieren?*

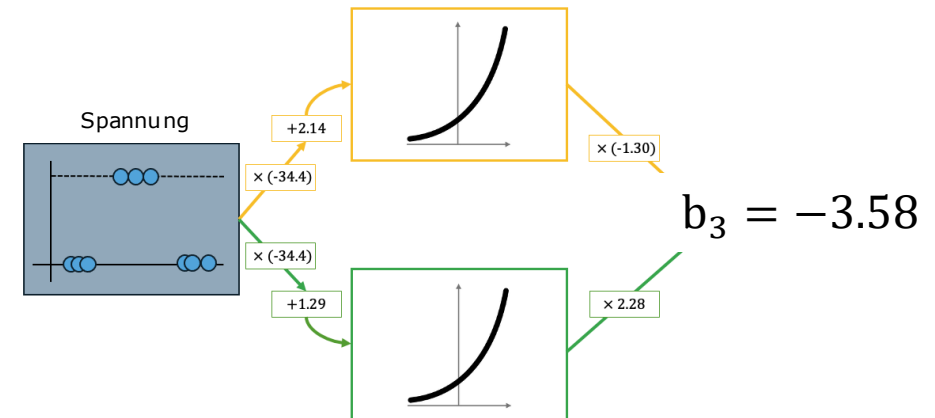
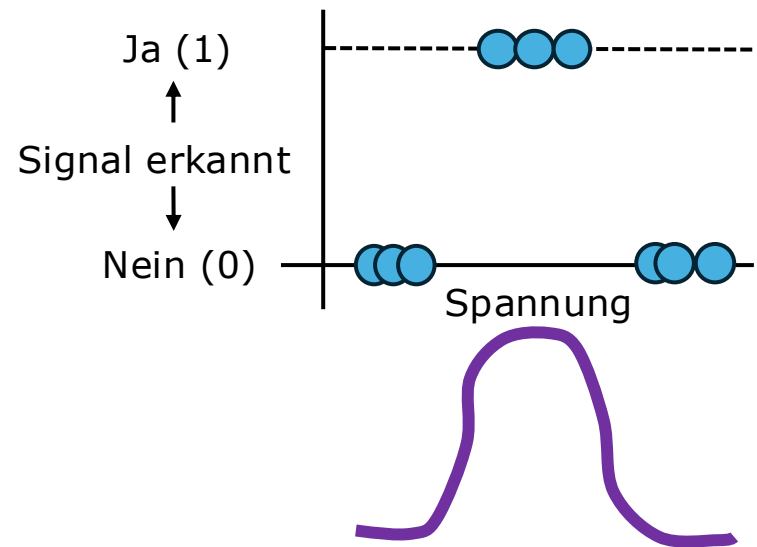
Das Lernziel



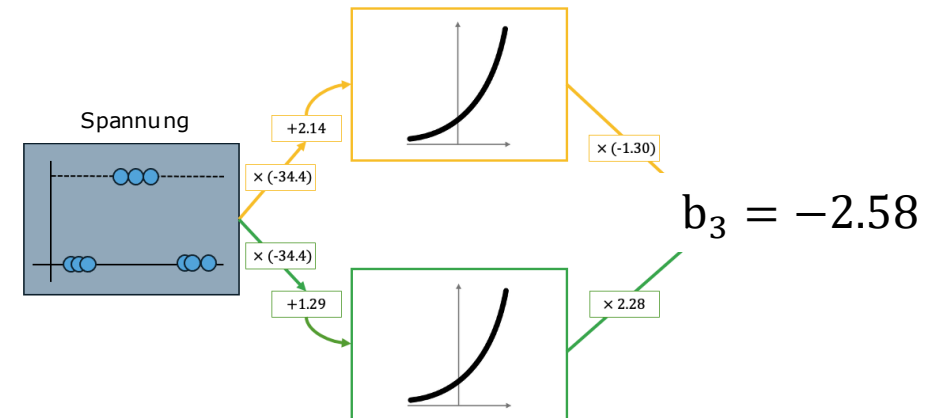
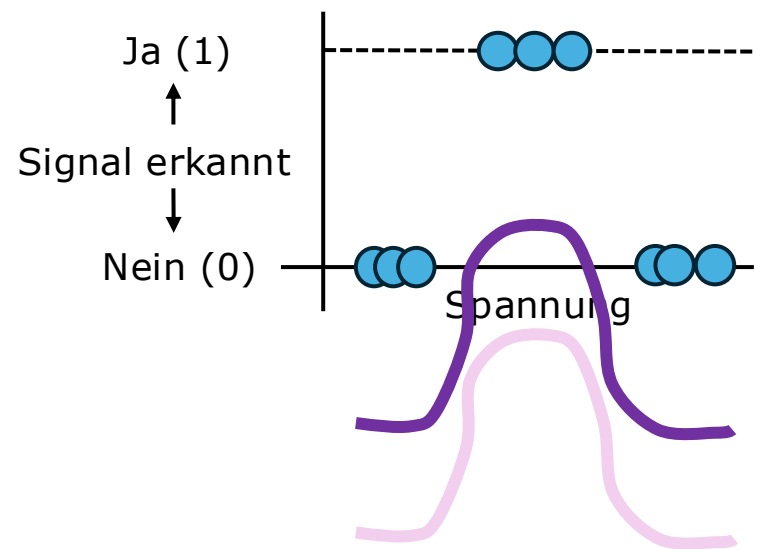
Stellen wir uns vor,
was mit unserer
Vorhersage
passiert, wenn wir
den **letzten Bias-Term** verändern
von diesem Wert ...



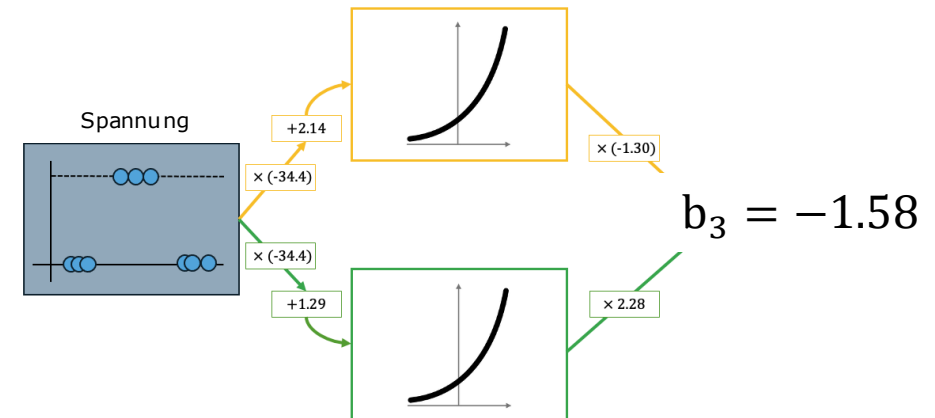
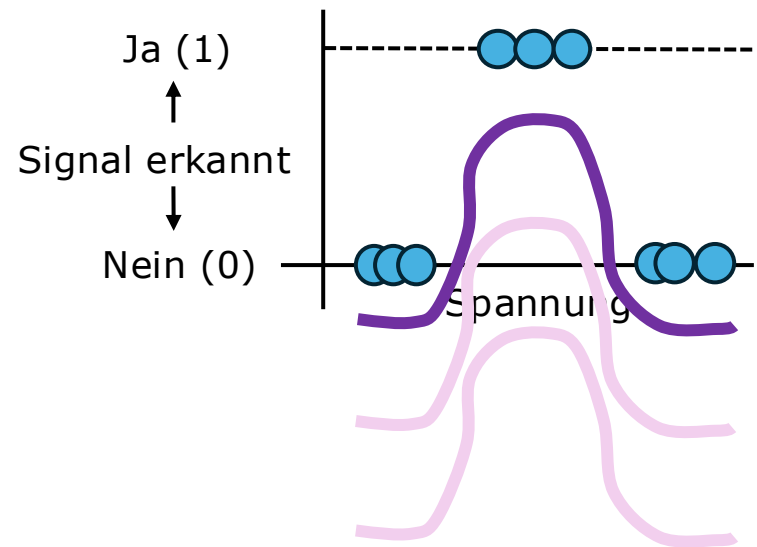
Das Lernziel



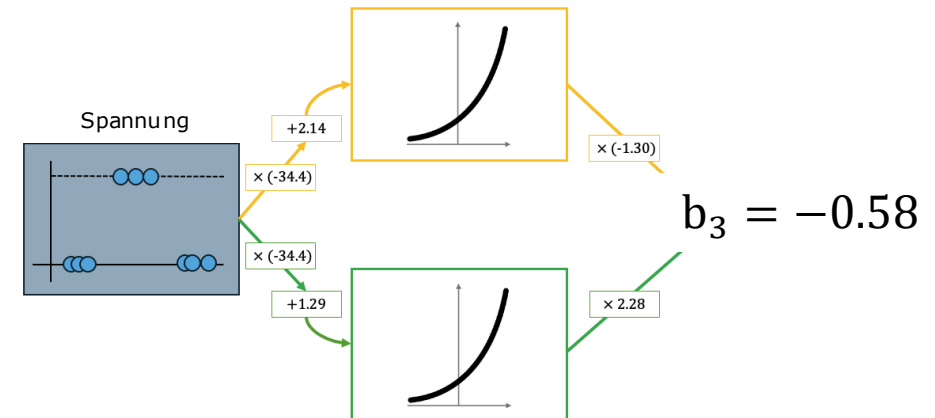
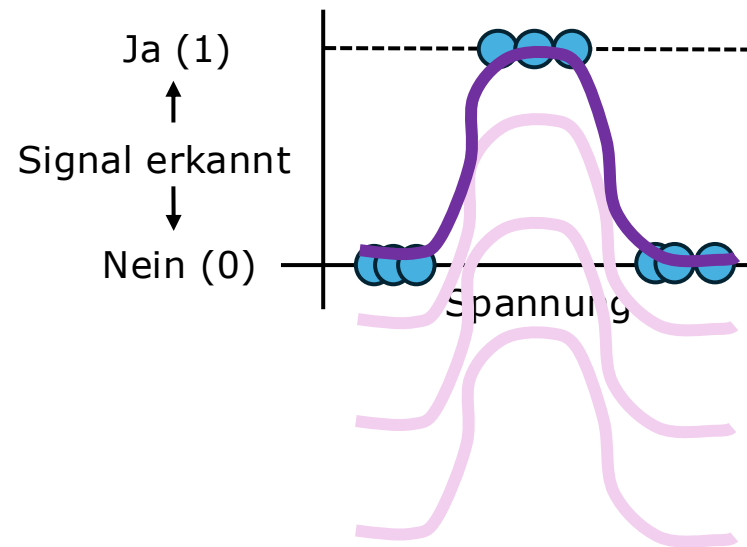
Das Lernziel



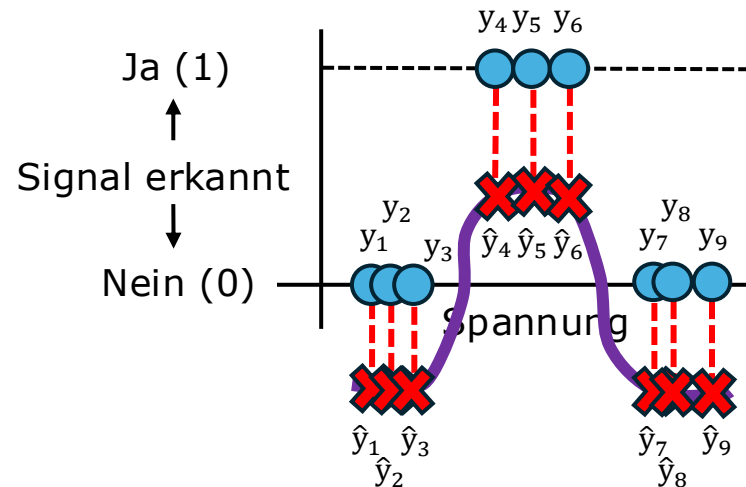
Das Lernziel



Das Lernziel



Das Lernziel



Ziel: Die **beste Kurve finden**, die unsere Daten möglichst gut beschreibt.

Dazu vergleichen wir unsere **Vorhersage** $\hat{y}$ mit dem **tatsächlichen Wert** y (Ground Truth). Wir berechnen also den **Fehler zwischen Vorhersage und Realität**.

Dieser Fehler wird durch eine **Loss-Funktion** (Verlustfunktion) quantifiziert. Typischerweise z. B.:

$$L = \frac{1}{2} \sum_{i=1}^9 (y_i - \hat{y}_i)^2$$

Das Lernziel

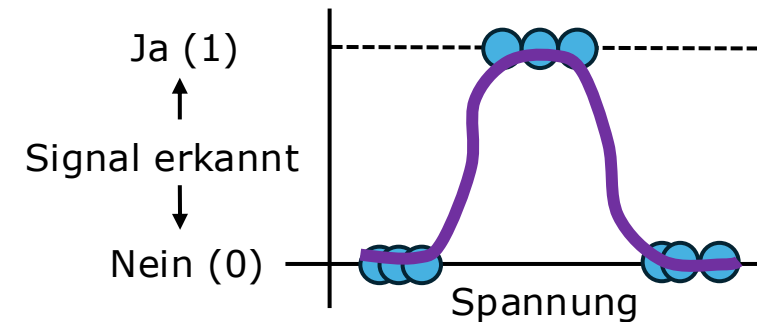
Viele Machine-Learning-Modelle werden trainiert, indem eine **Loss-Funktion** L minimiert wird

Beispiel:

$$L = \frac{1}{2} (y - \hat{y})^2$$

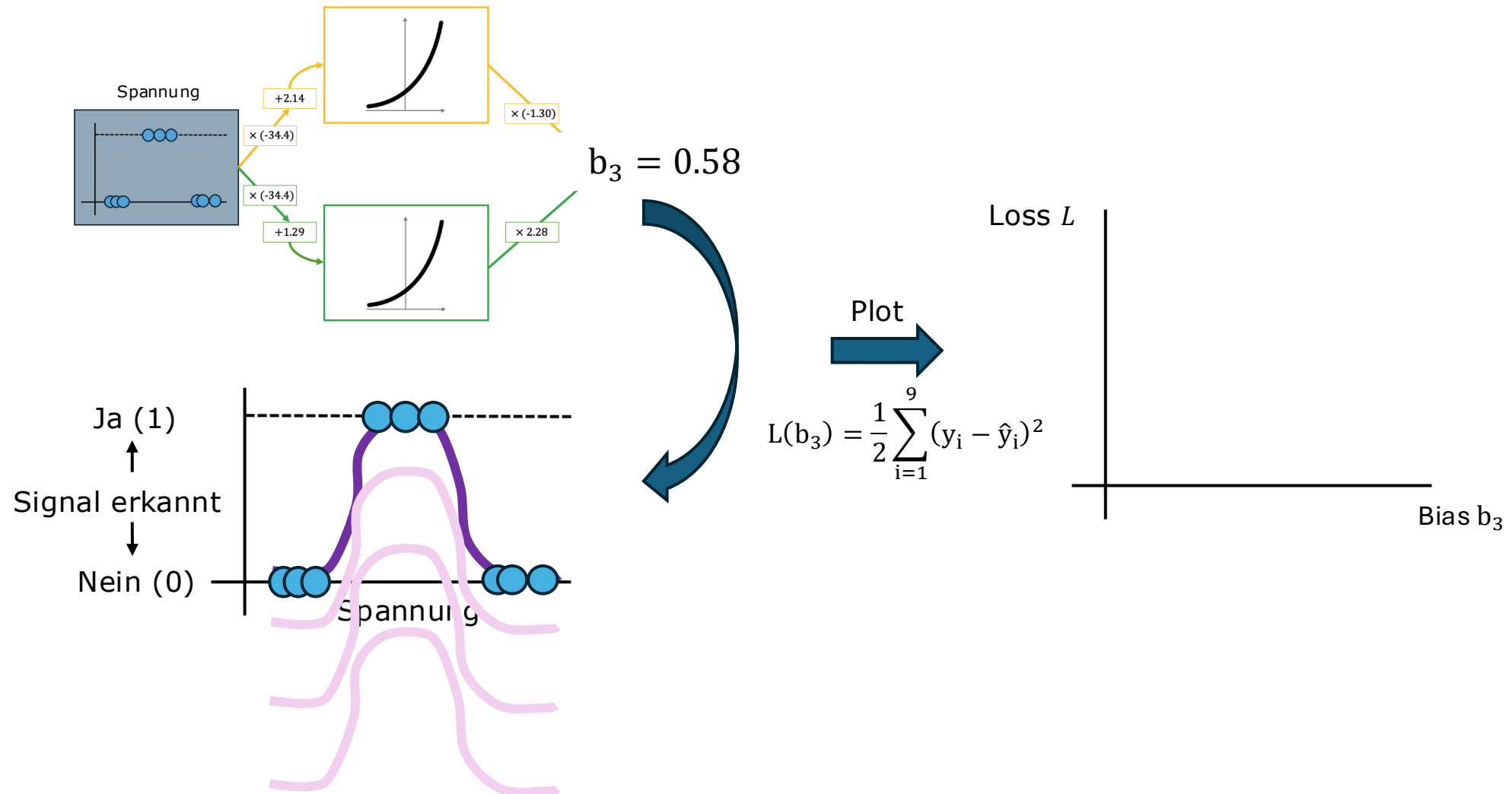
- y = Ground Truth (tatsächlicher Wert)
- $\hat{y}$ = Vorhersage des Modells
- **Ziel:** Passe die **Parameter** w **und** b so an, dass die **Loss** L minimiert wird →

$$\min_{w,b} L(y, \hat{y})$$

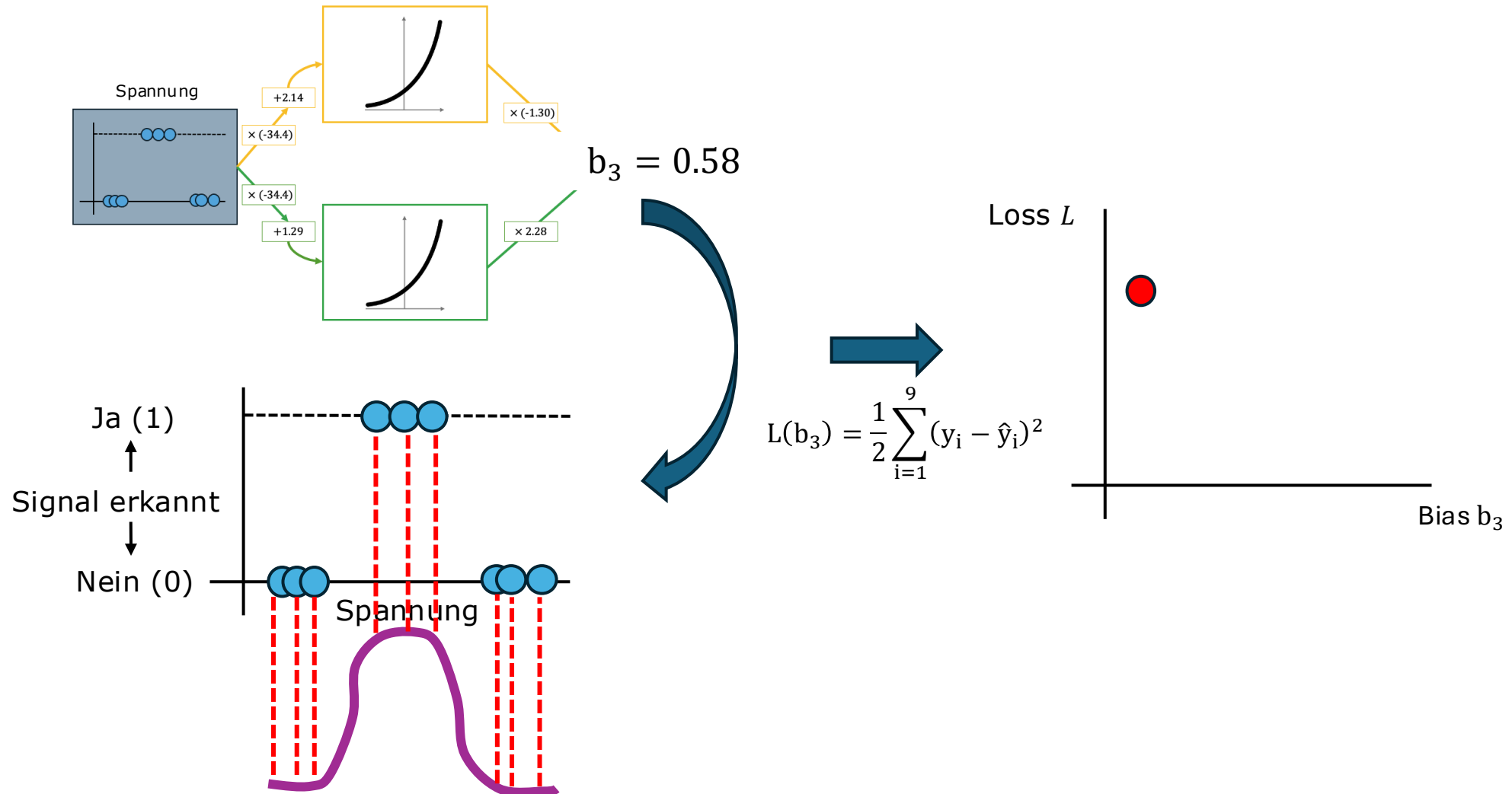


Die gute Nachricht ist:
 Ein **neuronales Netz**
kann auch komplexe,
nichtlineare Kurven an
die Daten anpassen

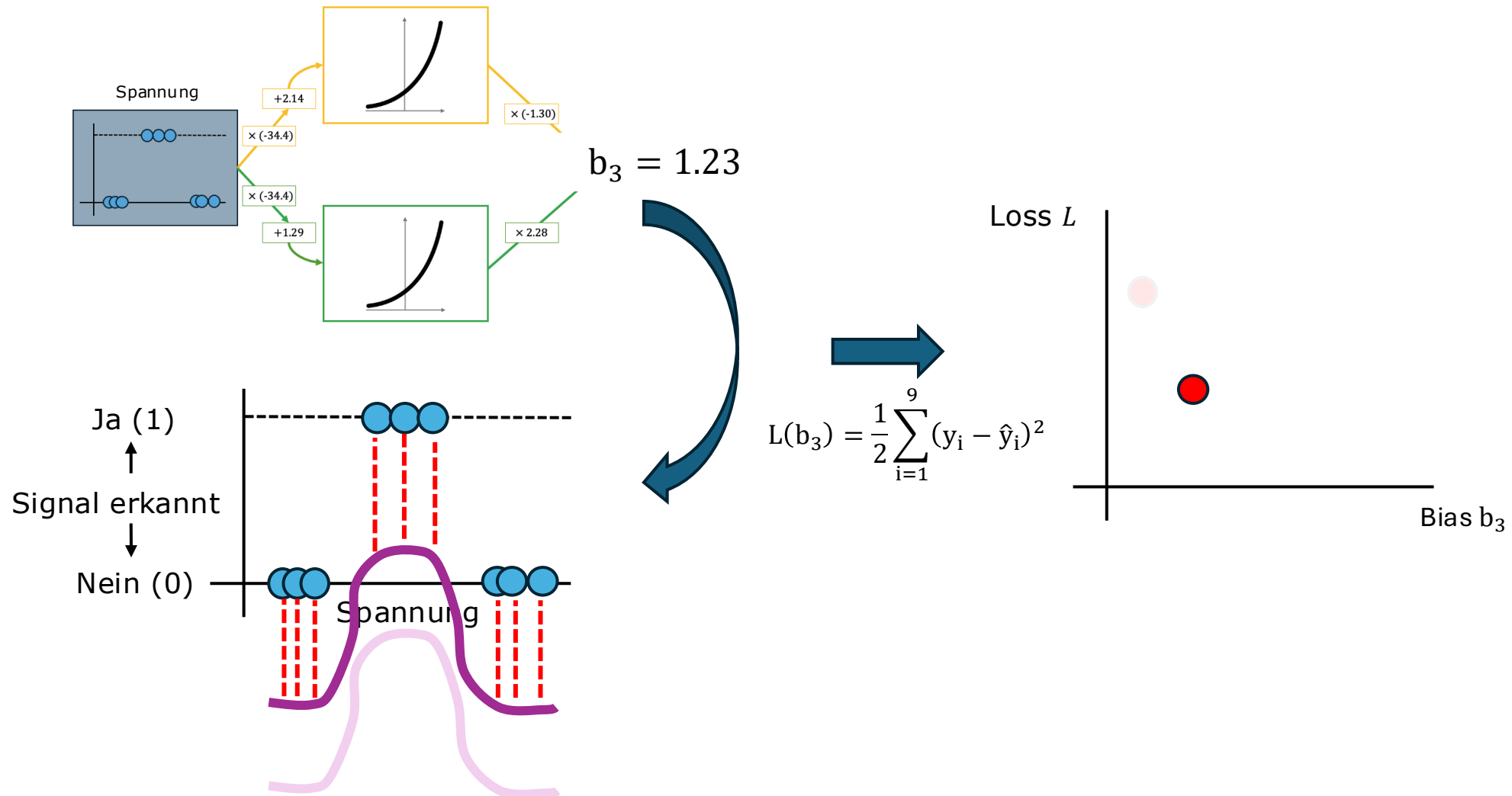
Das Lernziel



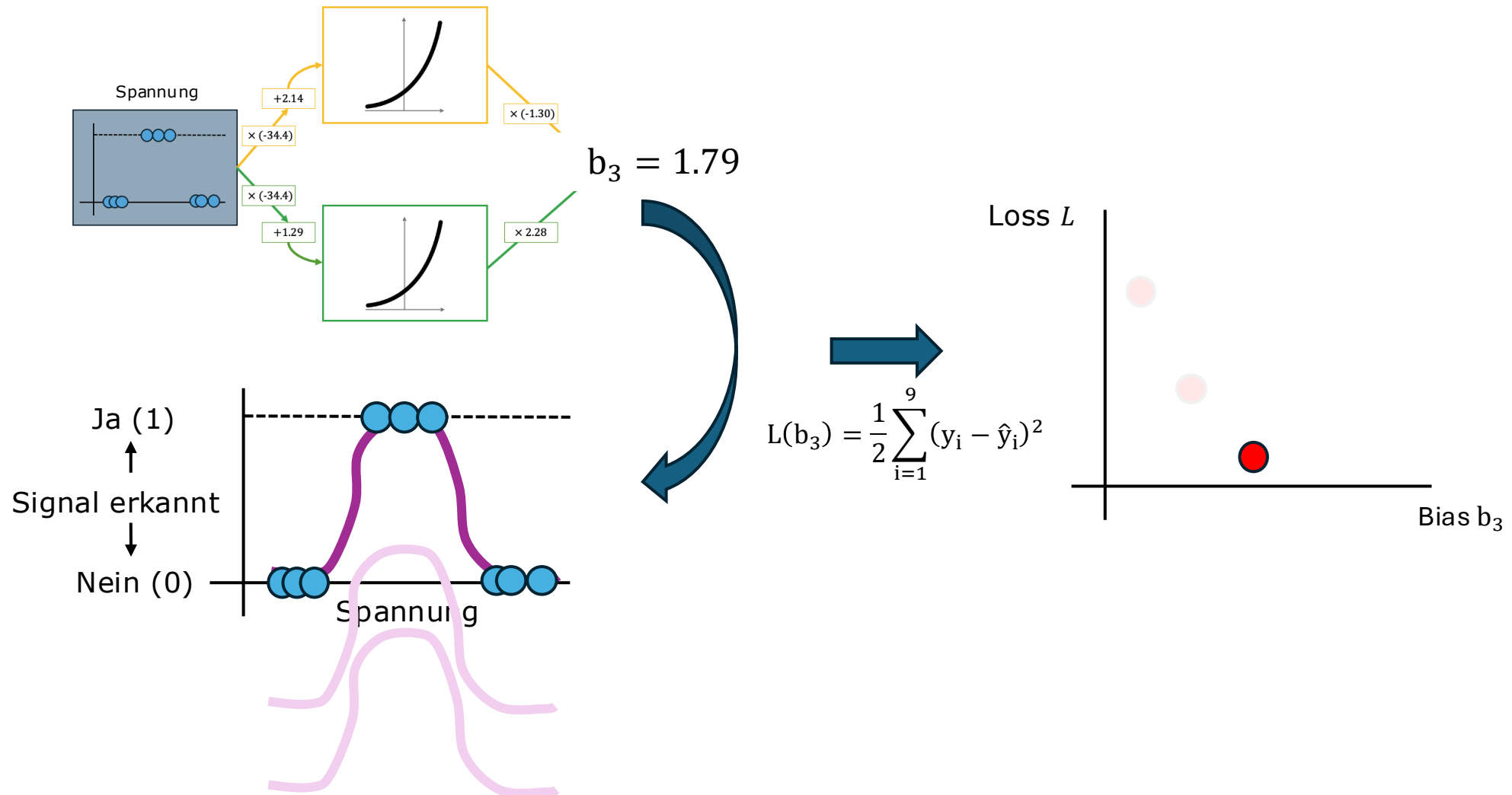
Das Lernziel



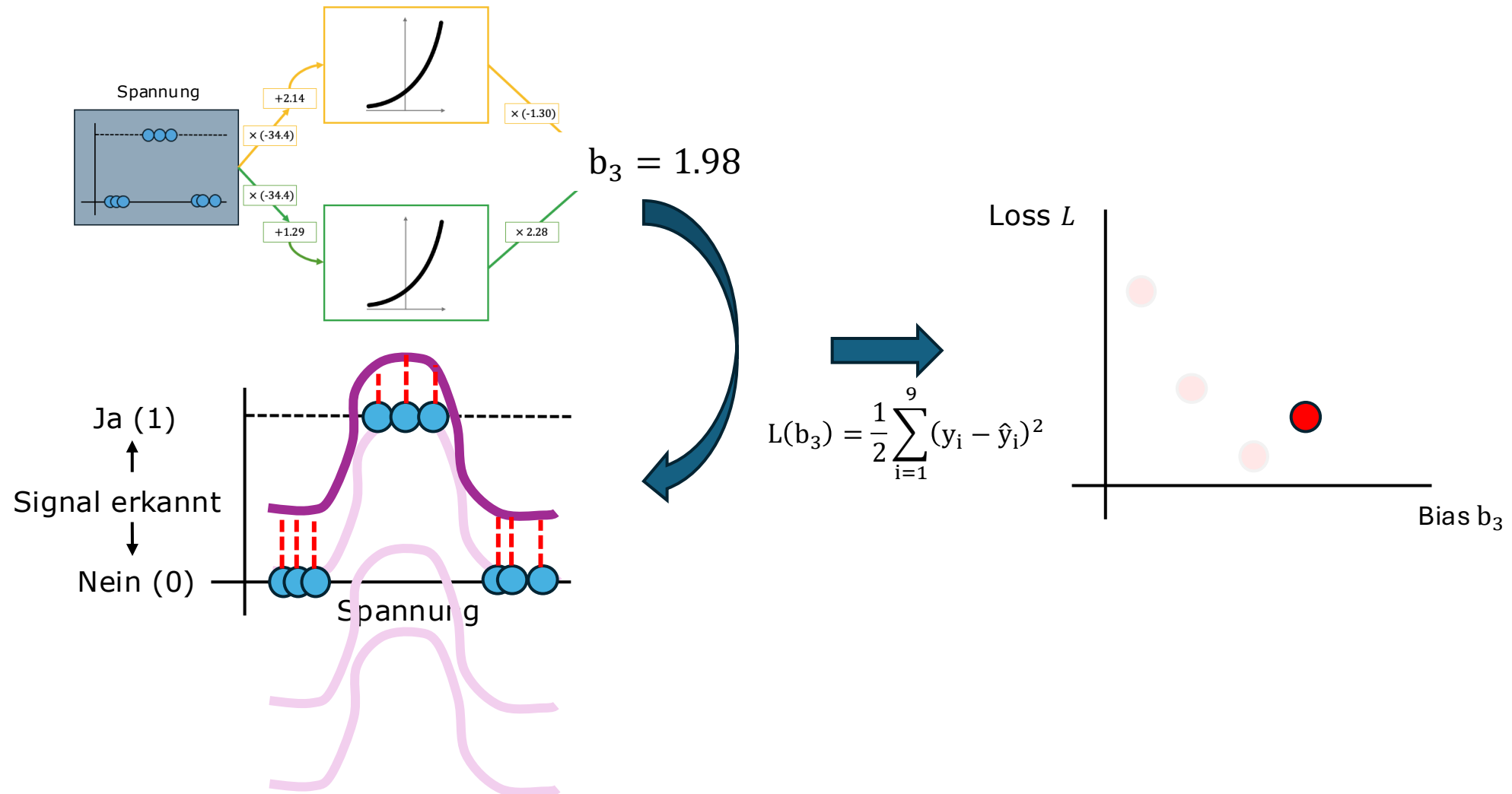
Das Lernziel



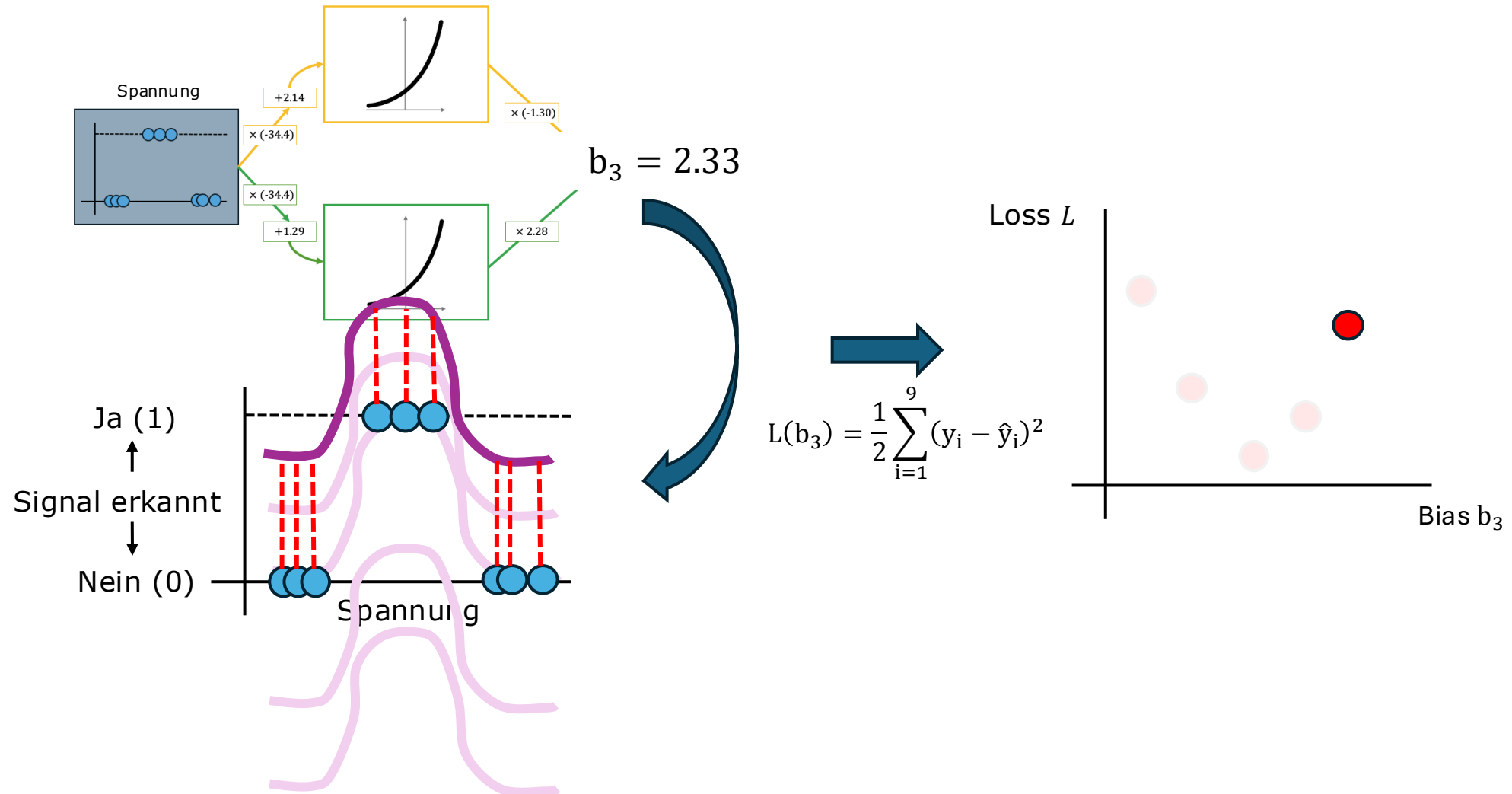
Das Lernziel



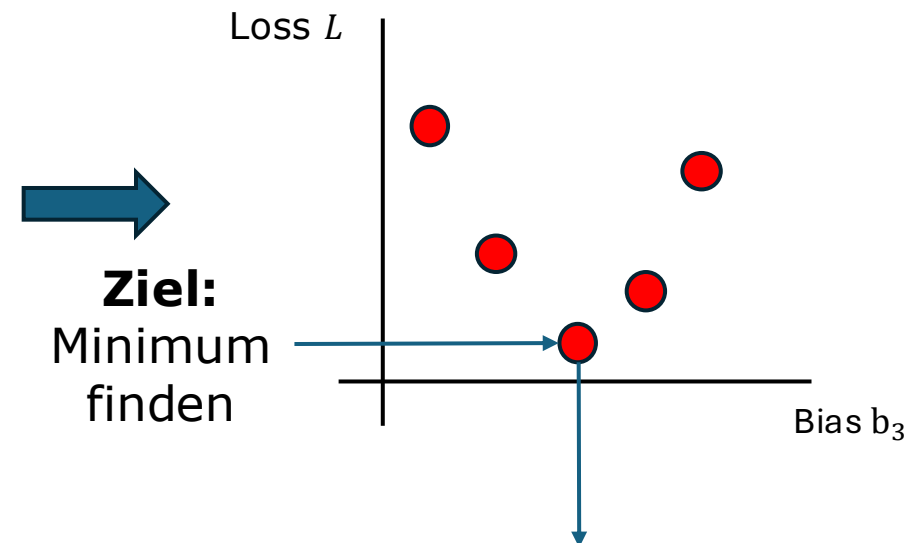
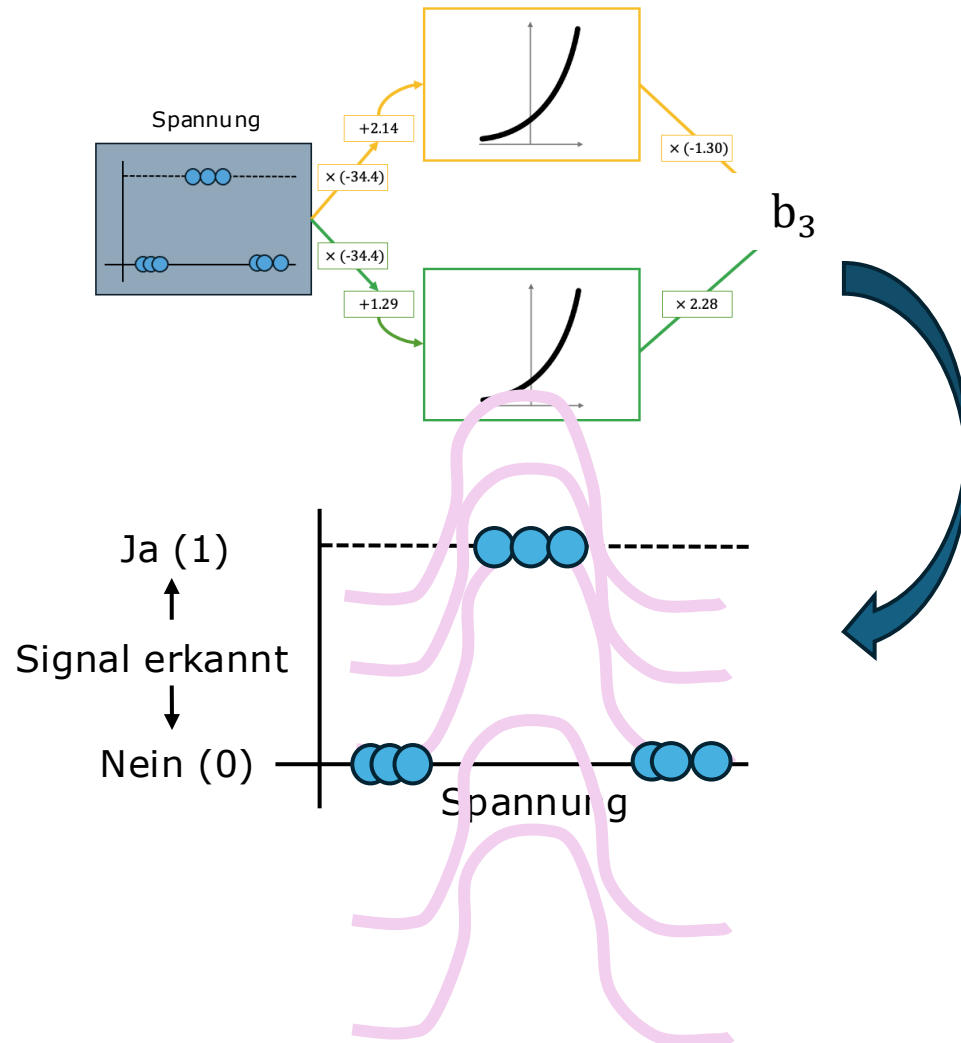
Das Lernziel



Das Lernziel



Das Lernziel



F: Wie kann ein Modell überhaupt lernen, welche Parameter die besten sind?

A: Genau hier kommt der **Gradientenabstieg** ins Spiel

Agenda

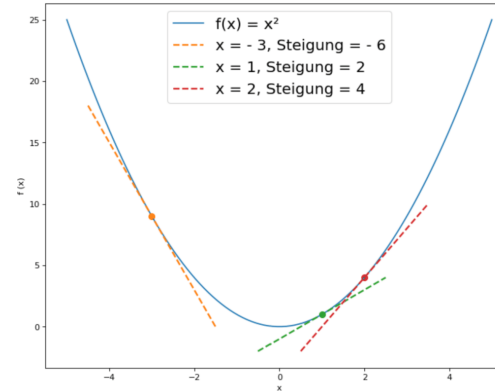
- Neuronale Netze
- **Gradientenabstieg**
- Beispiel

Was ist ein Gradient?

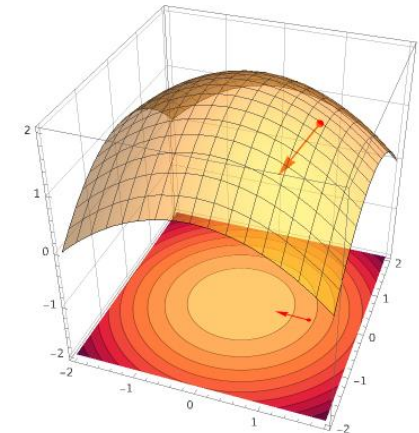
Der Gradient einer Funktion zeigt

- die Richtung des **stärksten Anstiegs**
- und die **Größe der Steigung**

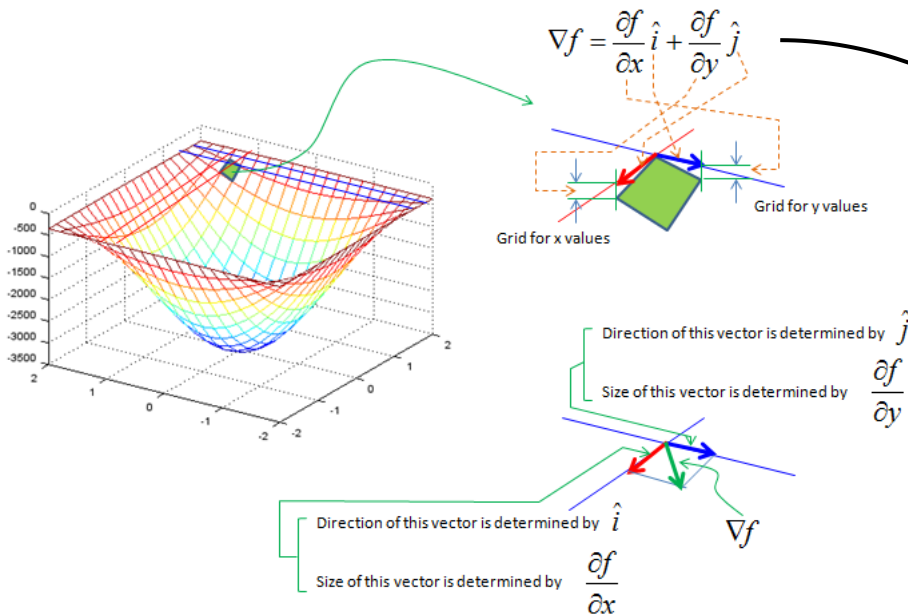
In **einer Dimension** entspricht der Gradient der **Ableitung** bzw. der **Steigung** der Funktion



<https://databasecamp.de/ki/stochastic-gradient-descent>



<https://demonstrations.wolfram.com/GradientIn2DAnd3D>



$$\nabla = \begin{pmatrix} \frac{\partial}{\partial x_1} \\ \vdots \\ \frac{\partial}{\partial x_n} \end{pmatrix}$$

Nabla-Operator (**partielle Ableitungen** nach den Ortskoordinaten)

$$\text{grad}(f)(x_0) = \nabla f(x_0) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x_0) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x_0) \end{pmatrix}$$

Funktion: $f(x, y) = e^{5x} + \sin(2y)$

Gradient: $\text{grad}(f)(x, y) = \nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x}(x, y) \\ \frac{\partial f}{\partial y}(x, y) \end{pmatrix} = \begin{pmatrix} 5e^{5x} \\ 2 \cos(2y) \end{pmatrix}$

Beispiel

<https://mathematica.stackexchange.com/questions/299799/creating-a-3d-gradient-vector-field-plot-in-mathematica>

Was ist Gradientenabstieg (Gradient Descent)?

Update-Regel (Gradientenabstieg):

$$w := w - \eta \frac{\partial L}{\partial w}$$

- η : **Lernrate (Learning Rate)**
- $\frac{\partial L}{\partial w}$: **Gradient der Loss-Funktion** bezüglich w

Idee:

- Bewege die Parameter in **Richtung des steilsten Abstiegs**, um die Loss L zu minimieren



<https://www.geeksforgeeks.org/machine-learning/what-is-gradient-descent/>

Gradientenabstieg

Der Gradient zeigt die Richtung des steilsten Anstiegs

- Wir gehen in die entgegengesetzte Richtung

$$w := w - \eta \frac{\partial L}{\partial w}$$

Update-Regel:

$$w_{n+1} = w_n - \eta \nabla f(w_n), n \geq 0$$

Wir erhalten eine monotone Folge

$$f(w_0) \geq f(w_1) \geq f(w_2) \geq \dots$$

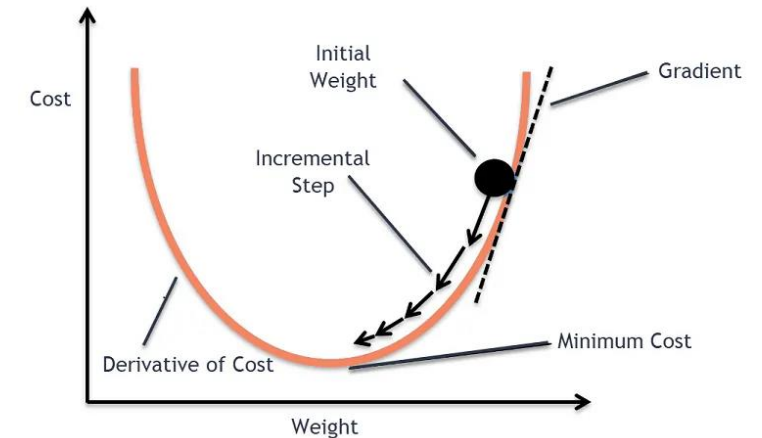
in der die Folge (w_n) gegen das gewünschte lokale Minimum konvergiert

Das können wir jetzt auf unsere **Loss-Funktion** (oder Kostenfunktion) anwenden:

$$L(\hat{y}, y) = \frac{1}{2} (y - \hat{y})^2$$

↓
↓

Tatsächlicher Wert
Vorhersage



<https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>

Gradientenabstieg Illustration

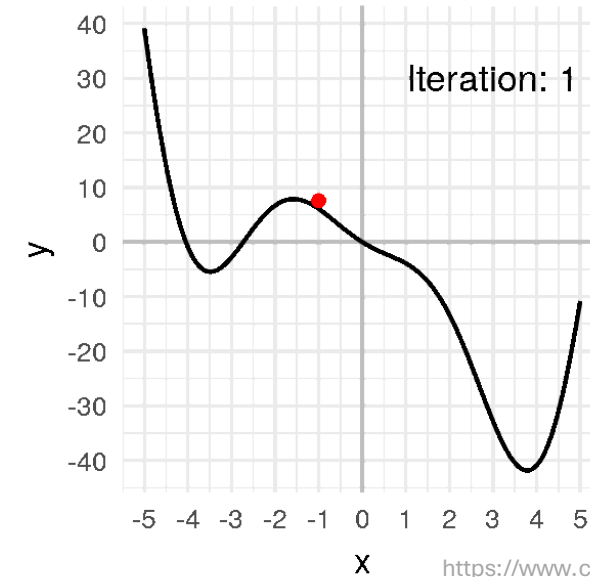
Update-Regel:

$$w := w - \eta \frac{\partial L}{\partial w}$$

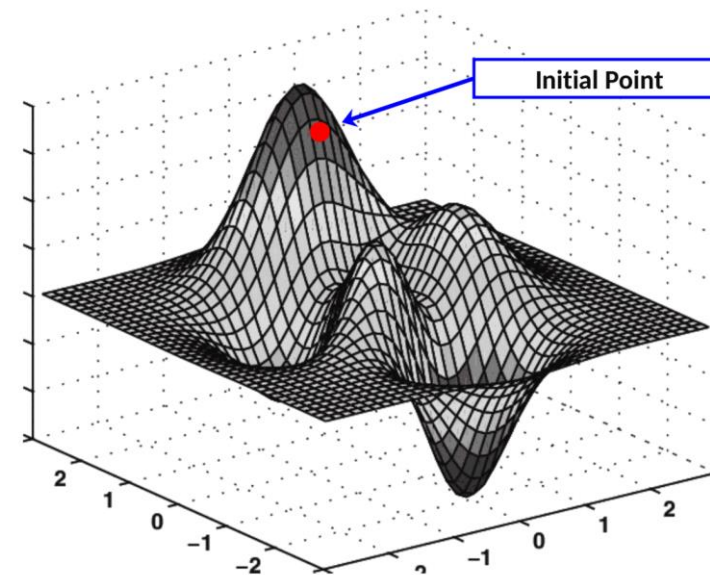
- η = Learning Rate
- $\frac{\partial L}{\partial w}$ = Gradient

Intuition

- Der **Gradient** zeigt die Richtung des **steilsten Anstiegs**
- Wir gehen **in die entgegengesetzte Richtung**
- Dadurch wird der Fehler **schrittweise kleiner**



<https://www.charlesbordet.com/en/gradient-descent/#what-to-do-in-case-of-local-minima>



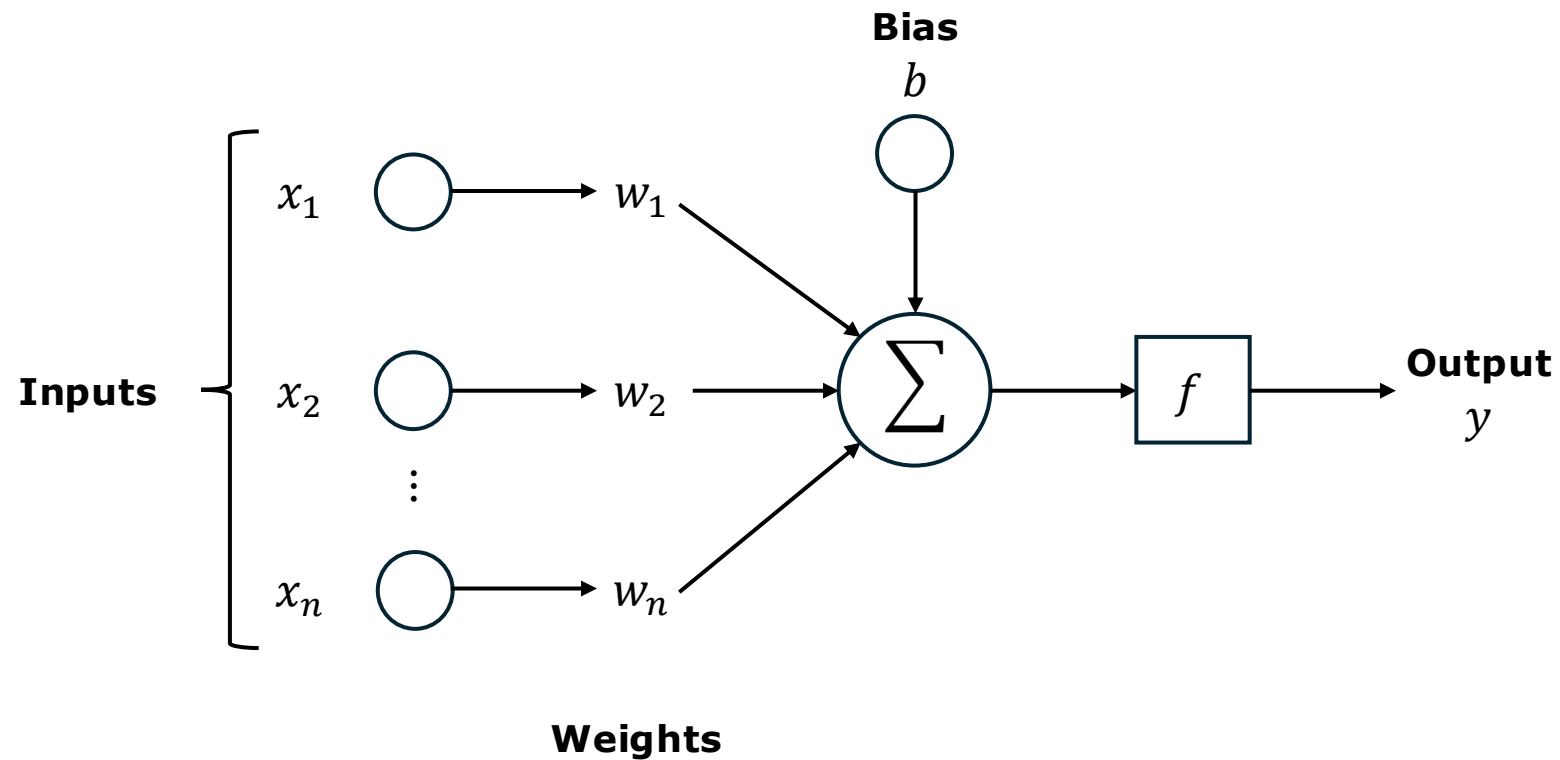
<https://pub.towardsai.net/deep-learning-from-scratch-in-modern-c-gradient-descent-670bc5889112>

Agenda

- Neuronale Netze
- Gradientenabstieg
- **Beispiel**

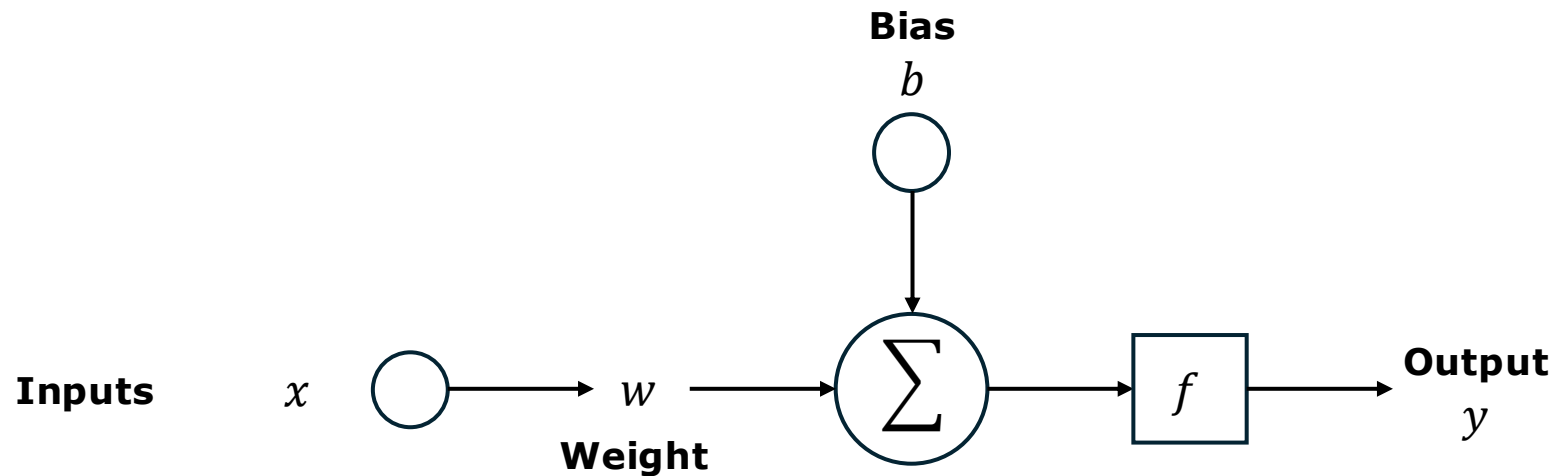
Gradientenabstieg Beispiel

Ein einfaches neuronales Netz sieht wie folgt aus:



Gradientenabstieg Beispiel

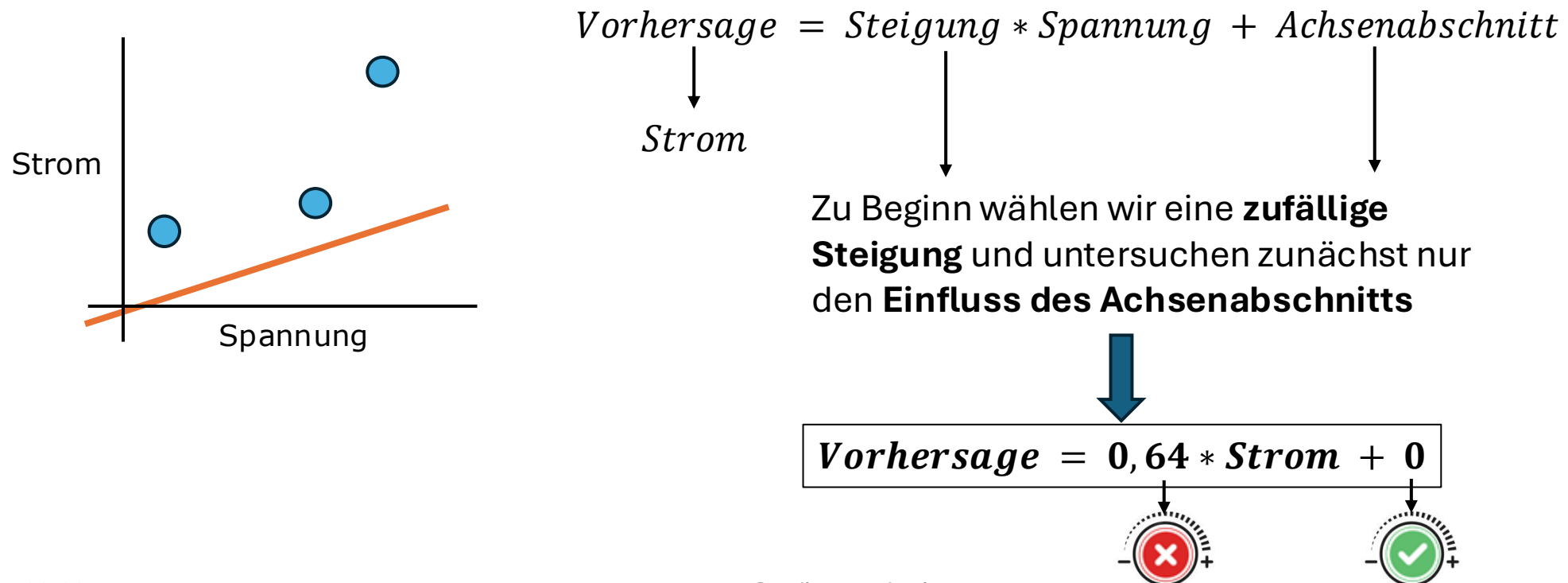
Wir betrachten nun ein stark vereinfachtes neuronales Netz mit nur einem Eingabewert und der linearen Aktivierungsfunktion ($f(x) = x$):



$\Rightarrow$ Dieses stark vereinfachte neuronale Netz entspricht mathematisch einer **linearen Regression**

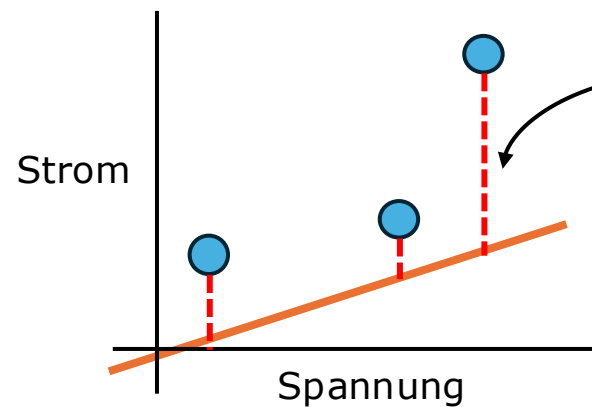
Gradientenabstieg Beispiel

Wir nehmen der Einfachheit halber an, dass wir nur **einen Input** haben und es sich um die lineare **Aktivierungsfunktion mit** $f(x) = x$ handelt:



Gradientenabstieg Beispiel

Jetzt können wir die **Residuenquadratsumme** (SQR) als unsere Loss Funktion verwenden:



$$\text{Vorhersage} = 0,64 * \text{Spannung} + 0$$

↓
 I

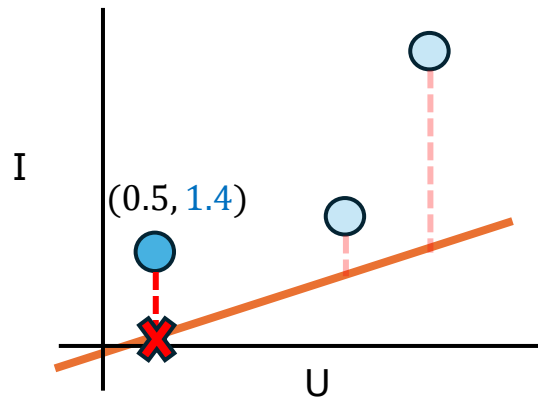
↓
 w

↓
 U

↓
 b

Gradientenabstieg Beispiel

Jetzt können wir die **Residuenquadratsumme** (SQR) als unsere Loss Funktion verwenden:

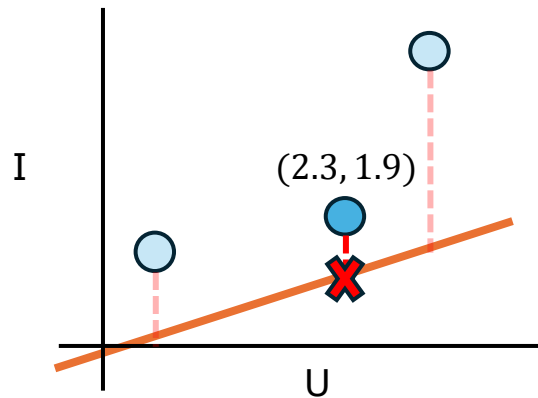


$$I = 0,64 * 0,5 + 0 = 0,32$$

$$Rest = 1,4 - 0,32 = 1,1$$

Gradientenabstieg Beispiel

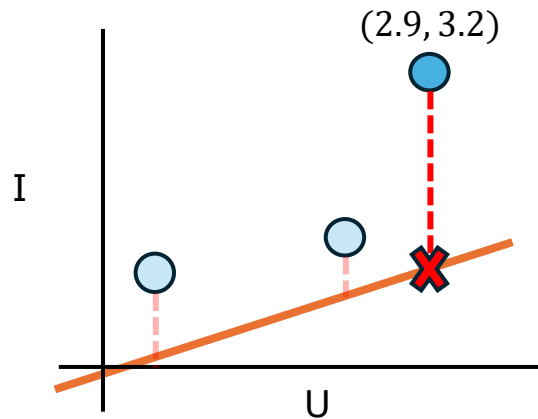
$$\text{Residuenquadratsumme (SQR)} = 1,1^2 + 0,4^2$$



... die zweite
Differenz ist **0,4** ...

Gradientenabstieg Beispiel

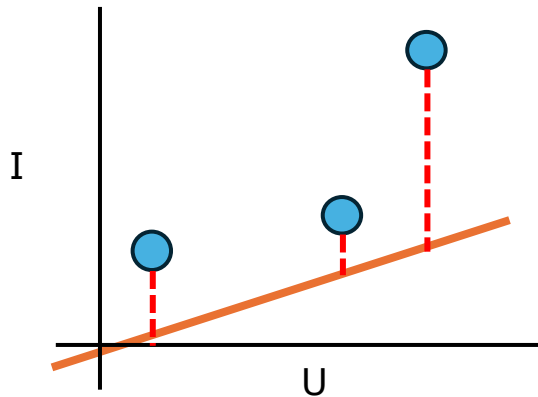
$$\textit{Residuenquadratsumme (SQR)} = 1,1^2 + 0,4^2 + 1,3^2$$



... die dritte
Differenz ist 1,3 ...

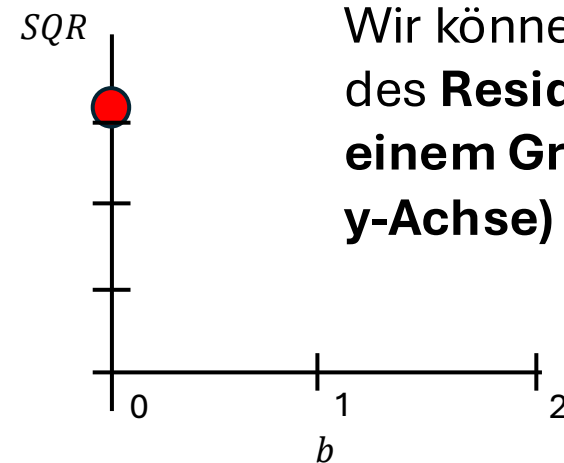
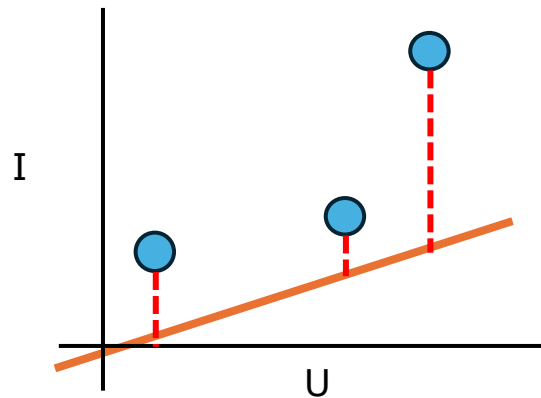
Gradientenabstieg Beispiel

$$\textit{Residuenquadratsumme (SQR)} = 1,1^2 + 0,4^2 + 1,3^2 = 3,1$$



Gradientenabstieg Beispiel

$$\text{Residuenquadratsumme (SQR)} = 1,1^2 + 0,4^2 + 1,3^2 = 3,1$$

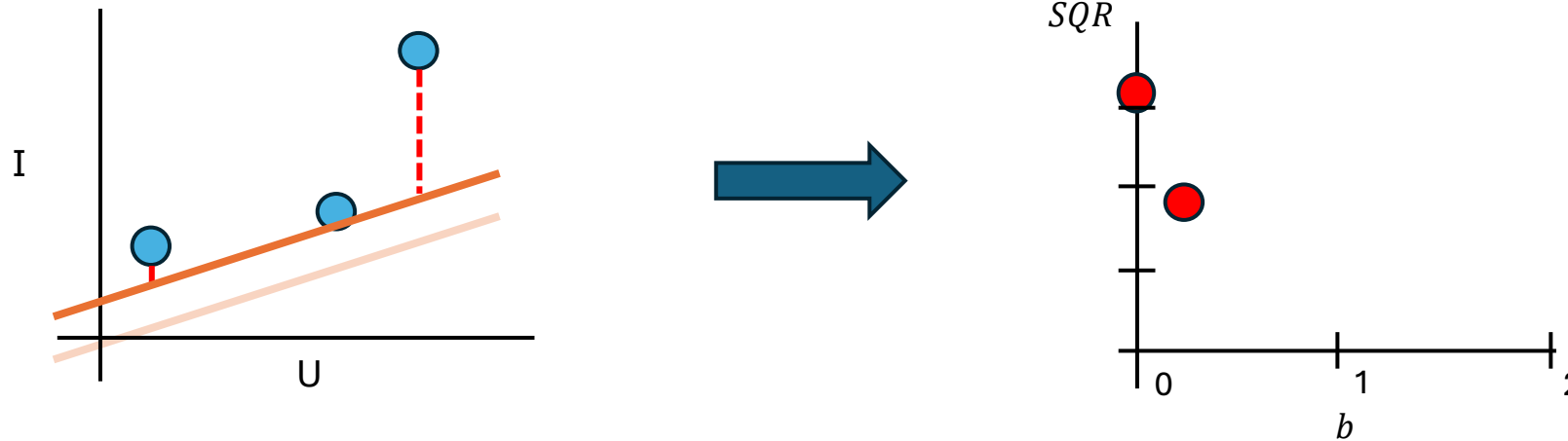


Wir können den Wert
des **Residuums** in
einem Graphen (auf der
y-Achse) eintragen ...

... und verschiedene
Achsenabschnittswerte
auf der **x-Achse**

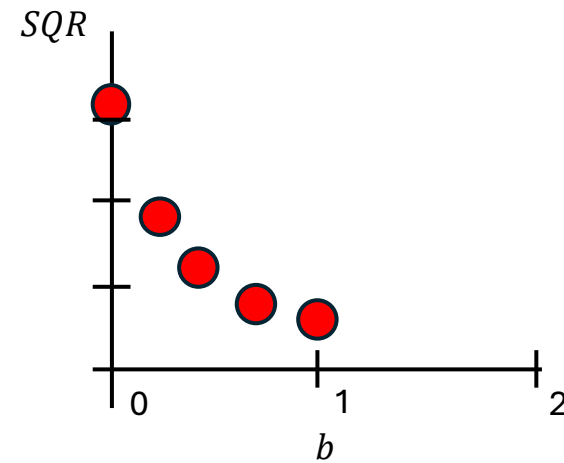
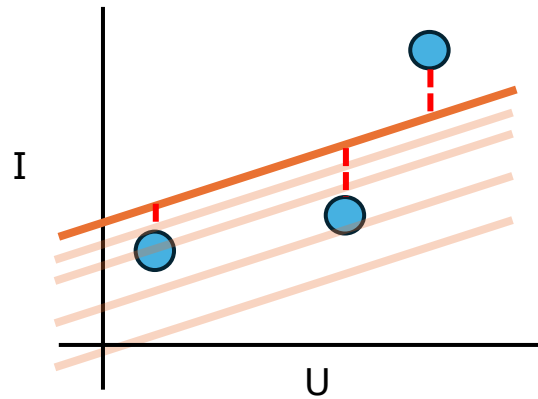
Gradientenabstieg Beispiel

Wir können jetzt mit verschiedenen Achsenabschnittswerten spielen und sie im Graphen darstellen, z.B. $b = 0,25$



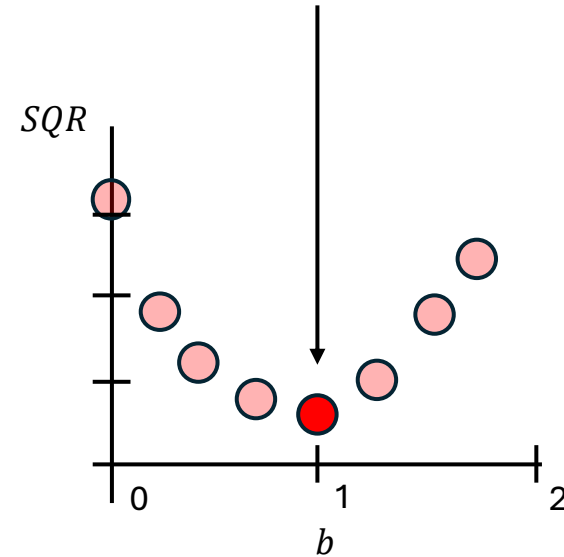
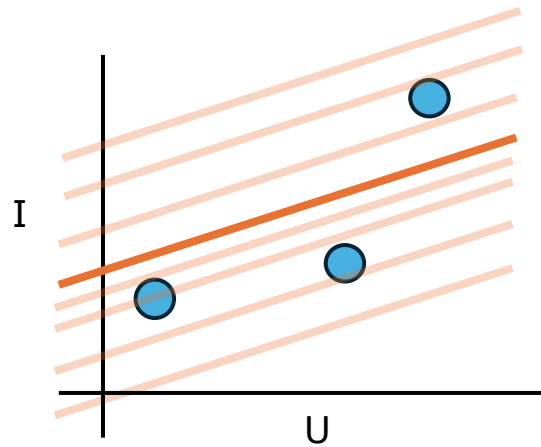
Gradientenabstieg Beispiel

USW ...



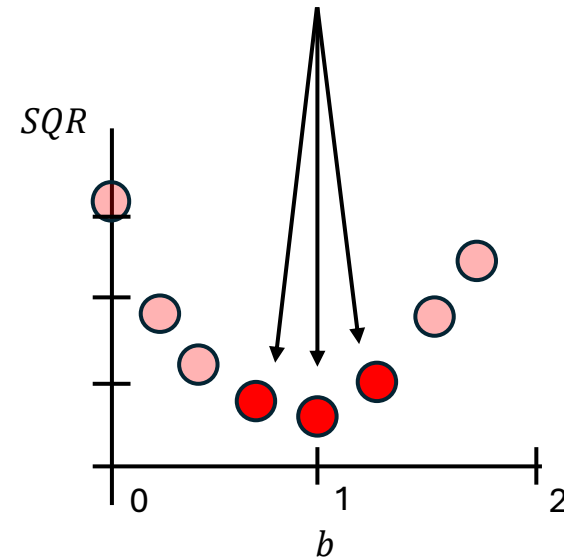
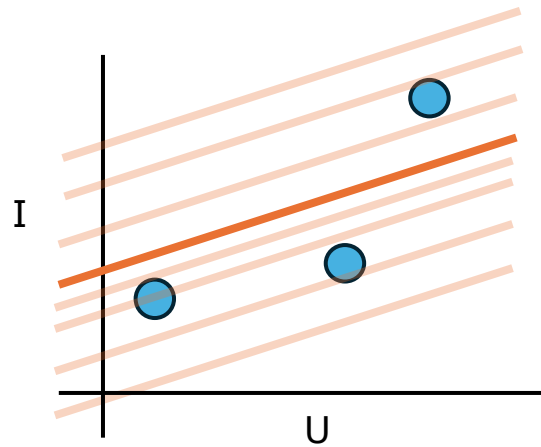
Gradientenabstieg Beispiel

Unter allen Punkten hat dieser den kleinsten Residuen-Wert

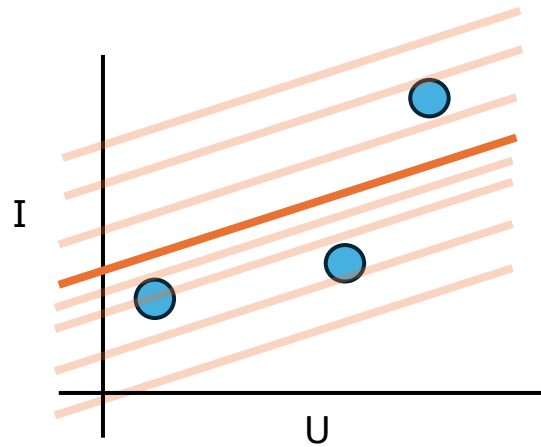


Gradientenabstieg Beispiel

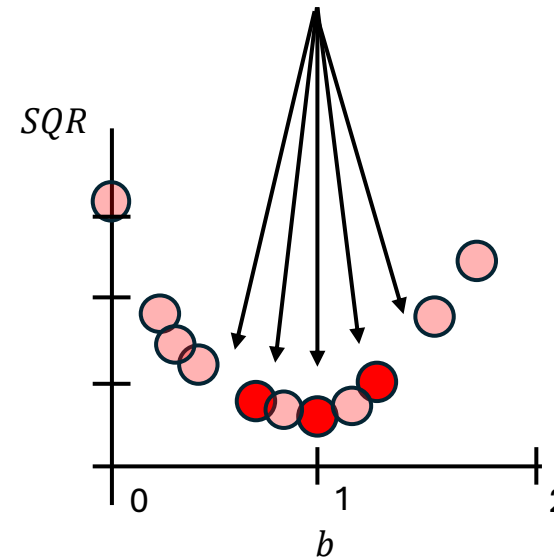
Was, wenn der minimale Wert
zwischen diesen Punkten liegt?



Gradientenabstieg Beispiel

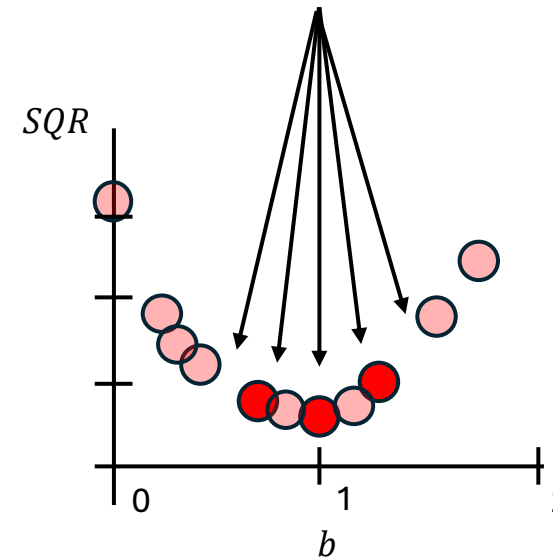
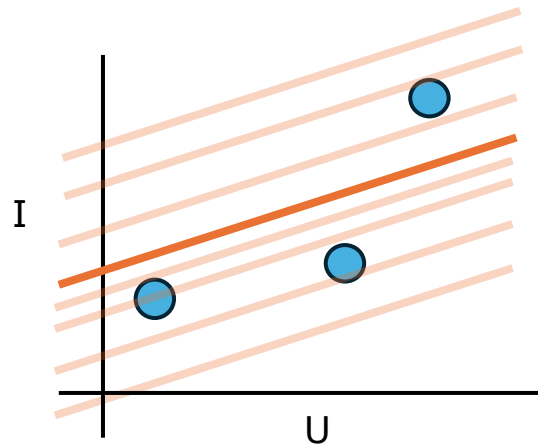


Viele Parameterwerte
auszuprobieren wäre möglich,
jedoch ineffizient



Gradientenabstieg Beispiel

Viele Punkte zu berechnen
wäre eine Möglichkeit, jedoch
eine sehr mühsame



→ Lösung Gradientenabstieg

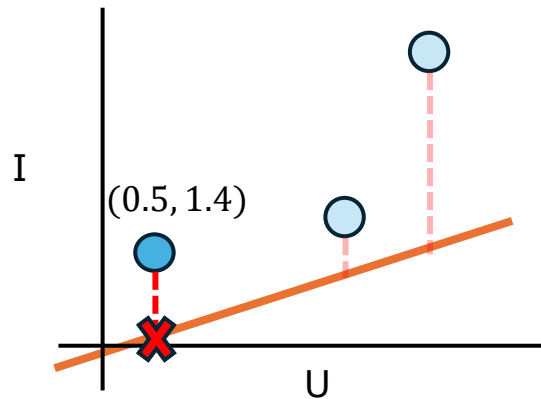
Gradientenabstieg Beispiel

Hierfür gehen wir kurz einen Schritt zurück:

$$SQR = (1,4 - (b + 0,64 * 0,5))^2$$



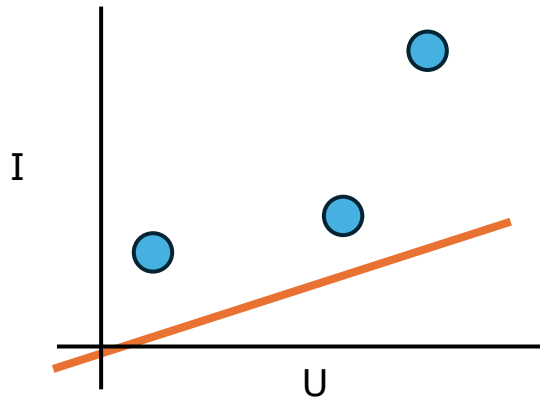
Wir können hier jetzt
jeden beliebigen Wert
einsetzen und eine
Vorhersage erhalten



Gradientenabstieg Beispiel

Wenn wir alle drei Punkte einsetzen, erhalten wir:

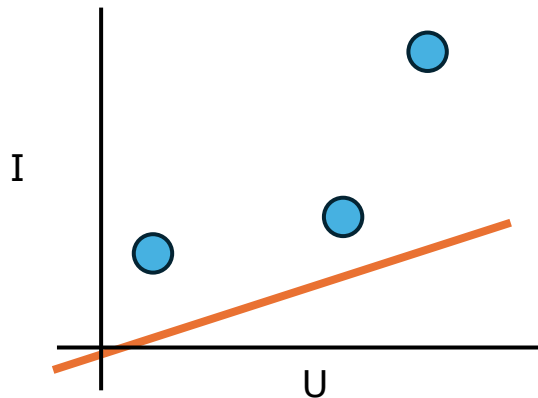
$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$



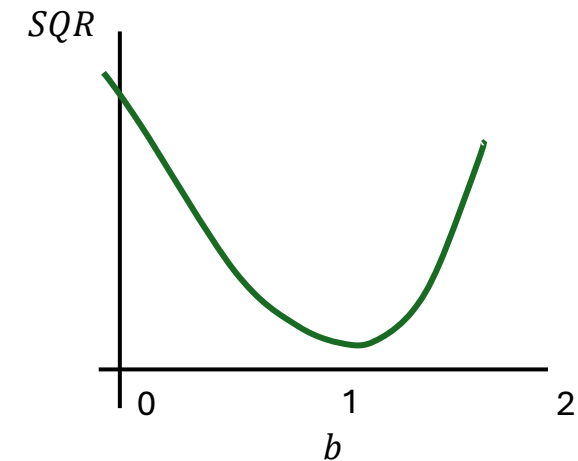
Gradientenabstieg Beispiel

Wenn wir alle drei Punkte einsetzen, erhalten wir:

$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$



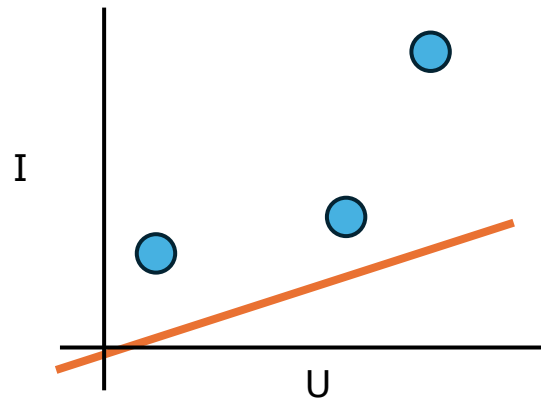
Jetzt können wir jeden beliebigen Wert einsetzen und erhalten diese **Kurve für SQR**



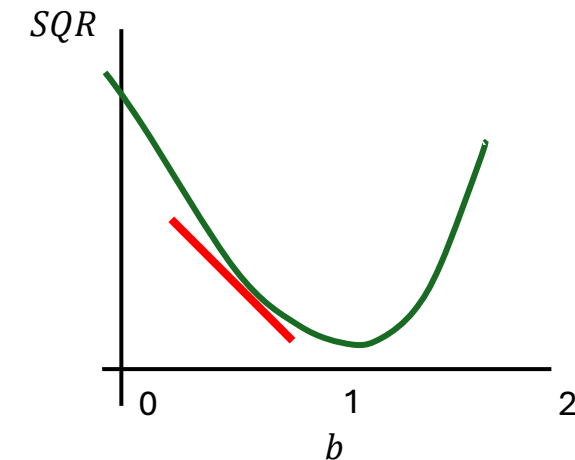
Gradientenabstieg Beispiel

Wenn wir alle drei Punkte einsetzen, erhalten wir:

$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$



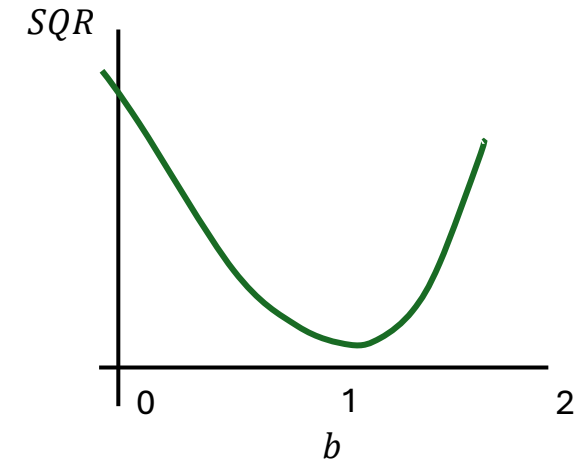
... und wir können die Ableitung dieser Funktion bestimmen und somit die Steigung an jedem beliebigen Punkt für den **Achsenabschnitt**



Gradientenabstieg Beispiel

$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$

Bestimmen wir nun die
Ableitung von SQR nach
dem Achsenabschnitt b



Gradientenabstieg Beispiel

$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$

$$\frac{d}{db} SQR = \frac{d}{db} (1,4 - (b + 0,64 * 0,5))^2 + \frac{d}{db} (1,9 - (b + 0,64 * 2,3))^2 + \frac{d}{db} (3,2 - (b + 0,64 * 2,9))^2$$

Gradientenabstieg Beispiel

$$SQR = (1,4 - (b + 0,64 * 0,5))^2 + (1,9 - (b + 0,64 * 2,3))^2 + (3,2 - (b + 0,64 * 2,9))^2$$

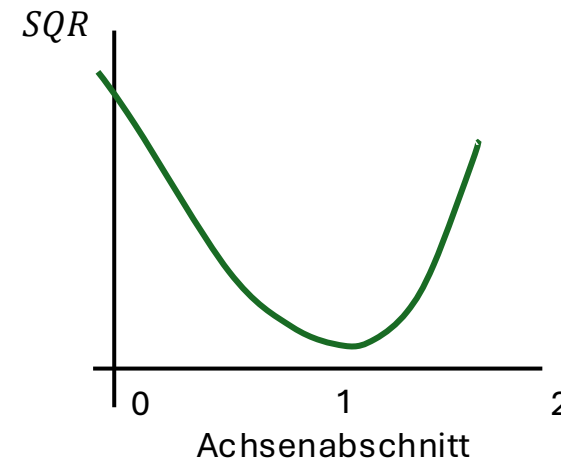
$$\frac{d}{db} SQR = \underbrace{\frac{d}{db} (1,4 - (b + 0,64 * 0,5))^2}_{\text{Kettenregel}} + \underbrace{\frac{d}{db} (1,9 - (b + 0,64 * 2,3))^2}_{\text{Kettenregel}} + \underbrace{\frac{d}{db} (3,2 - (b + 0,64 * 2,9))^2}_{\text{Kettenregel}}$$

$2(1,4 - (b + 0,64 * 0,5)) * (-1)$	$2(1,9 - (b + 0,64 * 2,3)) * (-1)$	$2(3,2 - (b + 0,64 * 2,9)) * (-1)$
------------------------------------	------------------------------------	------------------------------------

Gradientenabstieg Beispiel

$$\frac{d}{db}SQR = -2(1,4 - (b + 0,64 * 0,5)) - 2(1,9 - (b + 0,64 * 2,3)) - 2(3,2 - (b + 0,64 * 2,9))$$

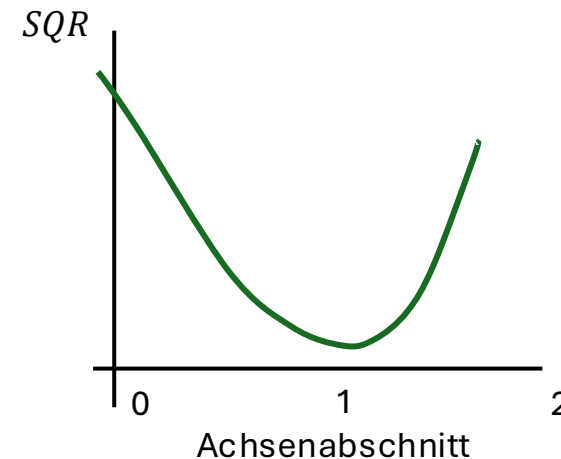
Mit der Ableitung können wir nun
 das **Gradientenabstiegsverfahren**
 verwenden, um das Minimum zu
 finden → Steigung = 0



Gradientenabstieg Beispiel

Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

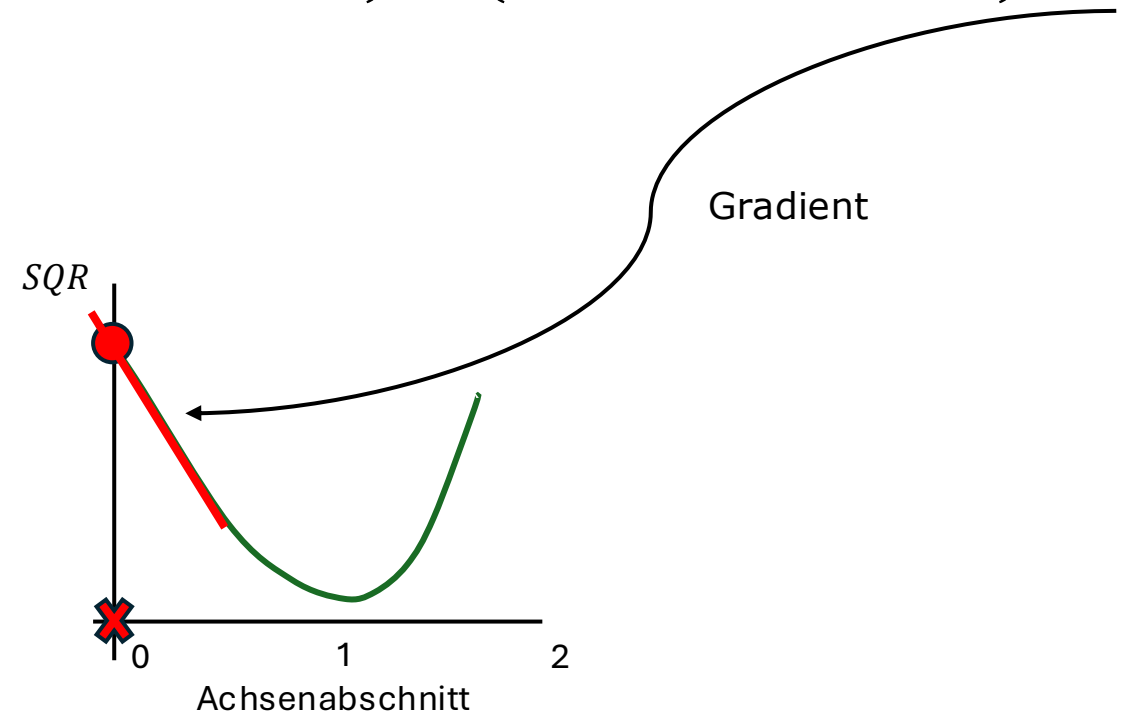
$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9))$$



Gradientenabstieg Beispiel

Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9)) = -5,7$$



Gradientenabstieg Beispiel

Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

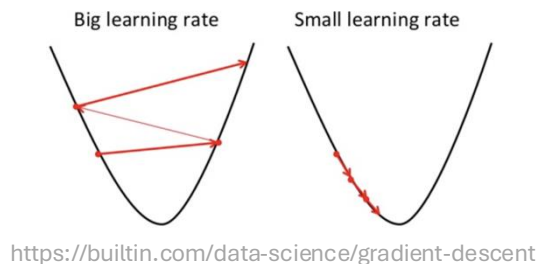
$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9)) = -5,7$$

$$b \leftarrow b - \eta \frac{\partial SQR}{\partial b}$$

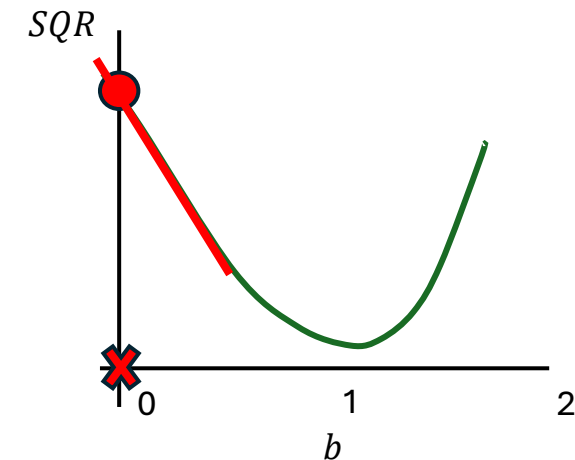
Zur Erinnerung:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Achsenabschnitt



Lernrate können wir frei bestimmen,
aber **Vorsicht**: zu groß $\rightarrow$ instabil, zu
klein $\rightarrow$ langsame Konvergenz



Gradientenabstieg Beispiel

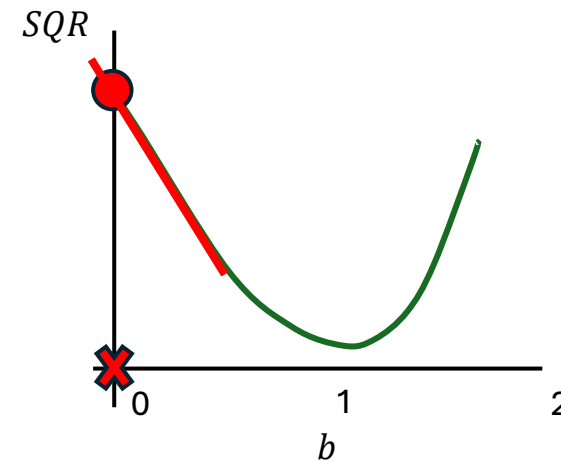
Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9)) = -5,7$$

Mit $\eta = 0,1$ erhalten wir:

$$b_{neu} \leftarrow 0 - 0,1 * (-5,7)$$

$$b_{neu} = 0,57$$



Gradientenabstieg Beispiel

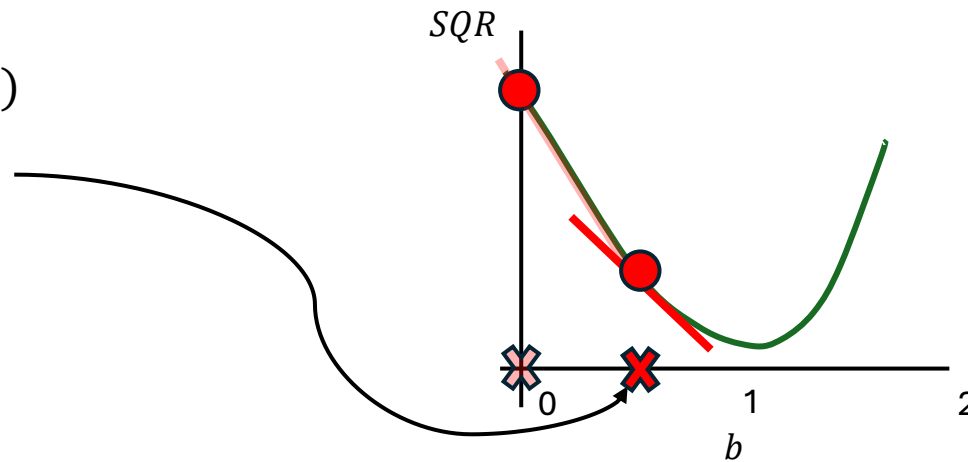
Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9)) = -5,7$$

Mit $\eta = 0,1$ erhalten wir:

$$b_{neu} \leftarrow 0 - 0,1 * (-5,7)$$

$$b_{neu} = 0,57$$



Gradientenabstieg Beispiel

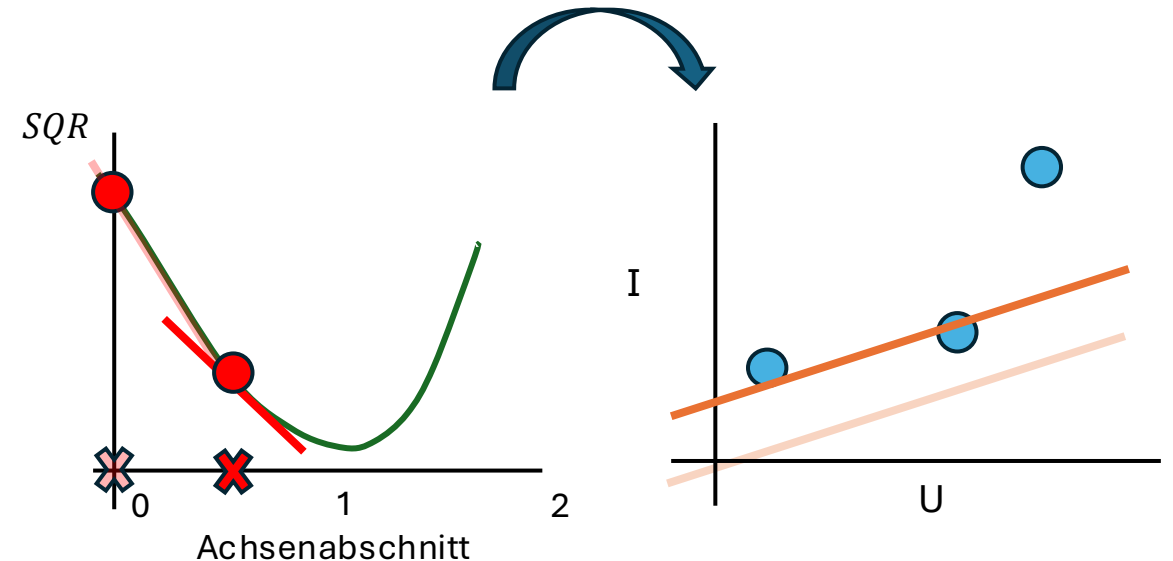
Bei $b = 0$ für den Achsenabschnitt anfangend, erhalten wir:

$$\frac{d}{db} SQR = -2(1,4 - (0 + 0,64 * 0,5)) - 2(1,9 - (0 + 0,64 * 2,3)) - 2(3,2 - (0 + 0,64 * 2,9)) = -5,7$$

Mit $\eta = 0,1$ erhalten wir:

$$b_{neu} \leftarrow 0 - 0,1 * (-5,7)$$

$$b_{neu} = 0,57$$



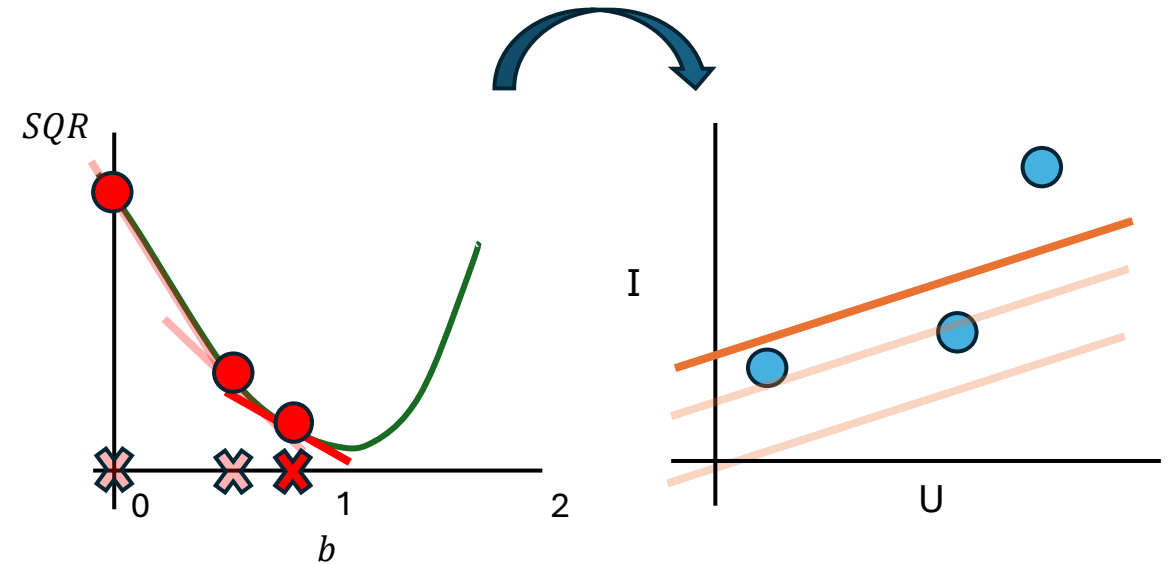
Gradientenabstieg Beispiel

Den neuen Wert für b einsetzend, erhalten wir:

$$\frac{d}{db} SQR = -2(1,4 - (0,57 + 0,64 * 0,5)) - 2(1,9 - (0,57 + 0,64 * 2,3)) - 2(3,2 - (0,57 + 0,64 * 2,9)) = -2,3$$

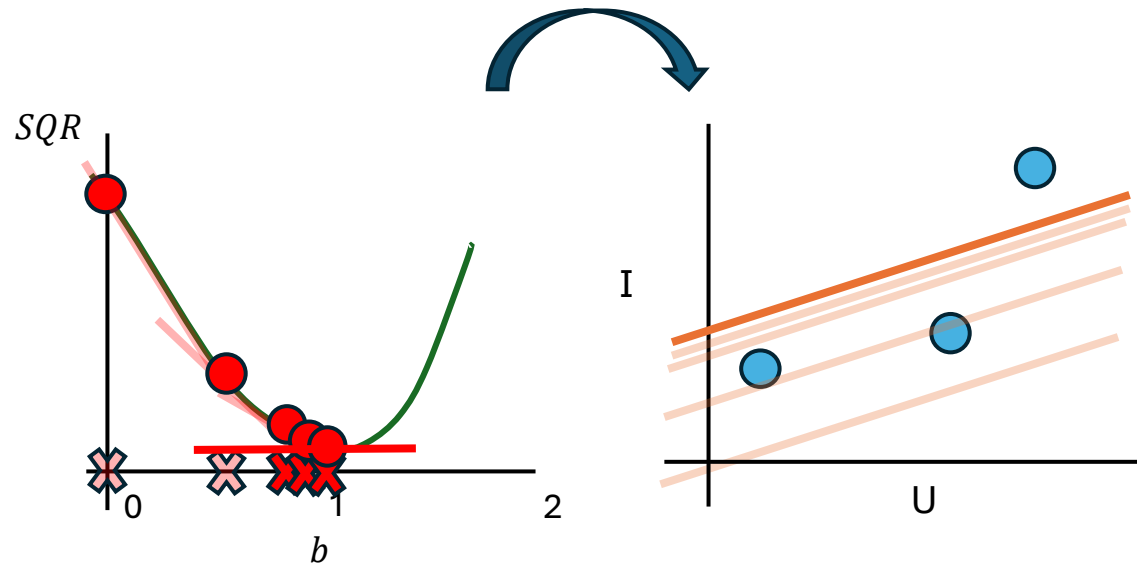
$$b_{neu} \leftarrow 0,57 - 0,1 * (-2,3)$$

$$b_{neu} = 0,8$$



Gradientenabstieg Beispiel

Der **Gradientenabstieg endet**, wenn eine **maximale Anzahl an Iterationen** erreicht ist oder die Verbesserung der **Loss-Funktion unter eine definierte Toleranzgrenze fällt**:

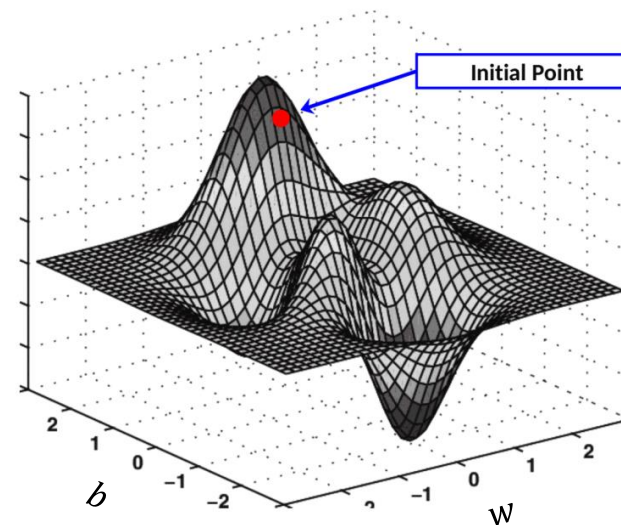
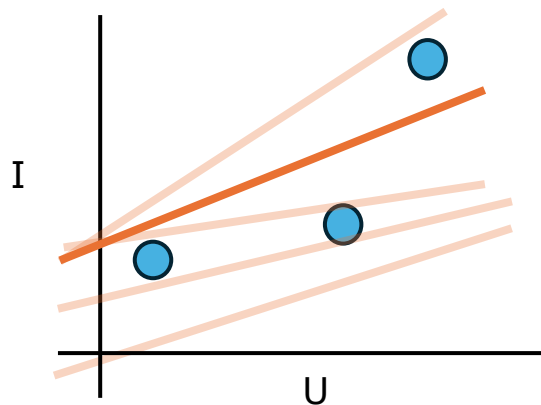


Gradientenabstieg Beispiel

Jetzt können wir das Ganze auch für die Steigung machen:

$$SQR = \underbrace{(1,4 - (b + w * 0,5))}^2 + (1,9 - (b + w * 2,3))^2 + (3,2 - (b + w * 2,9))^2$$

Jetzt beide Parameter $\theta = (w, b)$



<https://pub.towardsai.net/deep-learning-from-scratch-in-modern-c-gradient-descent-670bc5889112>

Gradientenabstieg Beispiel

Daraus ergibt sich, dass beide Ableitungen $\frac{d}{db} SQR$ und $\frac{d}{dw} SQR$ bestimmt werden müssen:

$$\nabla SQR(\theta) = \begin{bmatrix} \frac{\partial}{\partial b} SQR \\ \frac{\partial}{\partial w} SQR \end{bmatrix} \Rightarrow \text{Gradientenabstieg aktualisiert alle Modellparameter gleichzeitig}$$



nach allen Parametern

Gradientenabstieg Beispiel

Wenn man das ausrechnet:

$$\frac{\partial}{\partial b} SQR = -2(1,4 - (b + w * 0,5)) - 2(1,9 - (b + w * 2,3)) - 2(3,2 - (b + w * 2,9))$$

Nach zufälliger Initialisierung erfolgt die Aktualisierung nach der Update-Regel: $\theta_{neu} \leftarrow \theta_{alt} - 0,1 * \nabla SQR(\theta)$

$$\begin{pmatrix} b_{n+1} \\ w_{n+1} \end{pmatrix} = \begin{pmatrix} b_n \\ w_n \end{pmatrix} - 0,1 \begin{pmatrix} \frac{\partial SQR}{\partial b}(b, w) \\ \frac{\partial SQR}{\partial w}(b, w) \end{pmatrix}$$

$$\frac{\partial}{\partial w} SQR = -2 * 0,5(1,4 - (b + w * 0,5)) - 2 * 2,9(3,2 - (b + w * 2,9)) - 2 * 2,3(1,9 - (b + w * 2,3))$$



Selbst bei wenigen Datenpunkten ist dieses Vorgehen bereits aufwendig. **Stochastic Gradient Descent (SGD)** berechnet den Gradienten deshalb nur anhand eines zufällig gewählten Trainingsbeispiels statt des gesamten Datensatzes. → Das macht das Training großer neuronaler Netze überhaupt erst praktisch möglich.

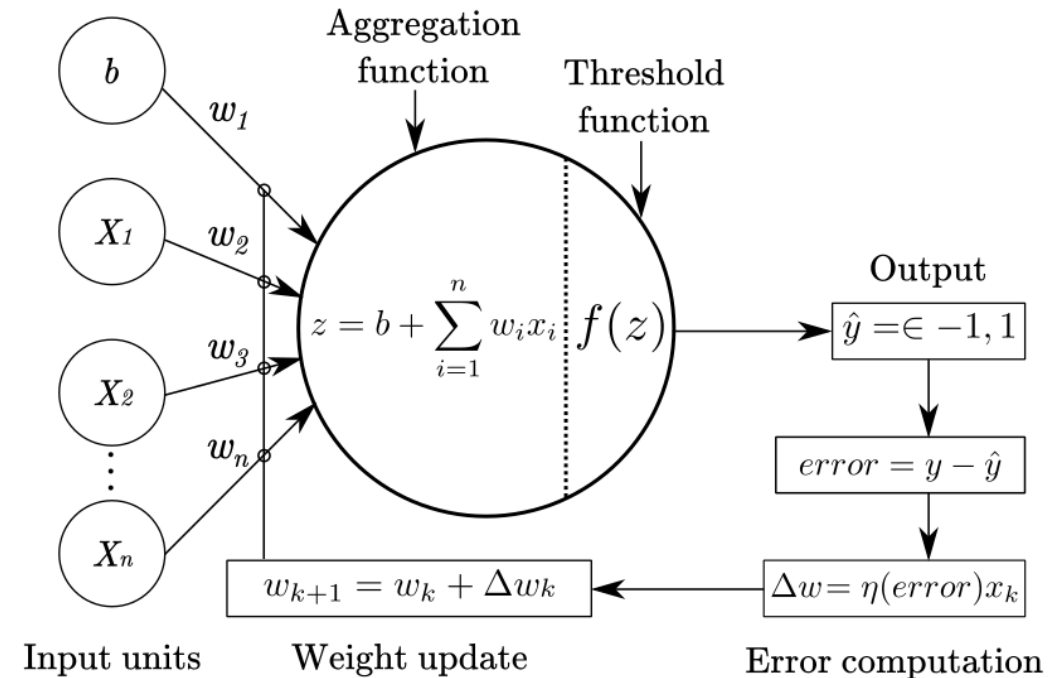
In unserem Beispiel optimieren wir nur zwei Parameter: w, b
In tiefen neuronalen Netzen optimieren wir jedoch einen **Vektor aller Parameter**: $\theta = (w_1, w_2, \dots, w_n, b_1, b_2, \dots, b_n)$
Gradient Descent wird dann angewendet auf: $\nabla_{\theta} L \rightarrow$ Prinzip bleibt **identisch**

Zusammenfassung



Gradientenabstieg

- Iterativer Optimierungsalgorithmus
- Minimiert eine Loss-Funktion
- Parameter werden schrittweise angepasst



<https://www.linkedin.com/pulse/deep-learning-1-ann-artificial-neural-network-jitender-malik-sup9c>

Weiterführende Quellen



Videos

- [Gradient Descent, Step-by-Step](#) ([StatQuest with Josh Starmer](#) ~ 24 min)
- [Gradient descent, how neural networks learn | Deep Learning Chapter 2](#) ([3Blue1Brown](#) ~ 20 min)
- [Backpropagation, Gradientenabstieg, Fehlersignal, Aktivität, Hyperparameter, Loss](#) ([Professor Joe](#) ~ 28 min)

Folien

- [2. Unconstrained Optimization / 2.1. Gradient Descent](#) (Lars Schmidt-Thieme)

Bücher

- [Hands-On Machine Learning with Scikit-Learn and PyTorch](#) (Aurélien Géron)
- [Math for Deep Learning](#) (Ronald T. Kneusel)

Kontakt



Mail:
konak@uni-potsdam.de

Folien von heute



<https://github.com/konakion/gradientenabstieg>